

長いだけのネクタイ (Just Long Neckties)

高谷悠太

問題概要

- 数列 $A_1, \dots, A_{N+1}$ と $B_1, \dots, B_N$ が与えられる
- 数列 A から要素を一つ除いて、残りの要素を並べ替えたものを $A'_1, \dots, A'_N$ とする
- 最初に A_k を除いたとき $\max\{A'_i - B_i, 0\}$ の最大値として考えられる最小値はいくつか？

サンプル 1

- $A = [4, 3, 7, 6], B = [2, 6, 4]$
- A_2 を除いて $A' = [4, 7, 6]$ とする
- このとき $\max\{A'_i - B_i, 0\}$ の最大値は2となる

小課題 1

- $N \leq 10$ なので $N!$ 回計算しても間に合う

小課題 1

- $N \leq 10$ なので $N!$ 回計算しても間に合う
- 取り除く要素を固定する
- A' の並べ方を固定する
- $\max\{A'_i - B_i, 0\}$ の最大値を求める

小課題 1

- $N \leq 10$ なので $N!$ 回計算しても間に合う
- 取り除く要素を固定 $\cdot \cdot \cdot$ N 通り
- A' の並べ方を固定 $\cdot \cdot \cdot$ $(N - 1)!$ 通り
- $\max\{A'_i - B_i, 0\}$ の最大値 $\cdot \cdot \cdot$ N 回計算

小課題 2

- $N \leq 2000$ なので計算回数を減らす必要あり

小課題 2

- $N \leq 2000$ なので計算回数を減らす必要あり
- 取り除く要素を固定 $\cdot \cdot \cdot$ N 通り
- A' の並べ方を固定 $\cdot \cdot \cdot$ $(N - 1)!$ 通り
- $\max\{A'_i - B_i, 0\}$ の最大値 $\cdot \cdot \cdot$ N 回計算

小課題 2

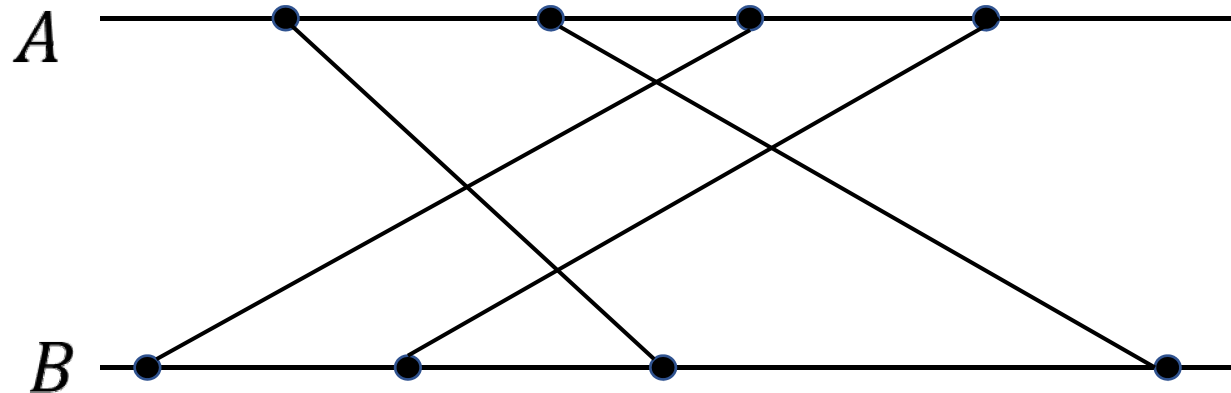
- $N \leq 2000$ なので計算回数を減らす必要あり
- 取り除く要素を固定 $\cdot \cdot \cdot N$ 通り
- A' の並べ方を固定 $\cdot \cdot \cdot \text{ ~~$(N-1)!$~~ 通り}$
- $\max\{A'_i - B_i, 0\}$ の最大値 $\cdot \cdot \cdot N$ 回計算

A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。

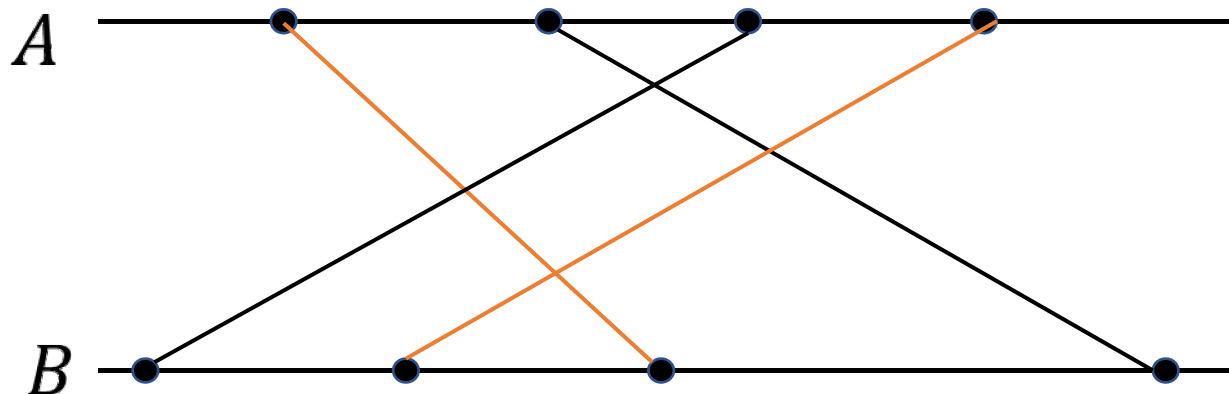
A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



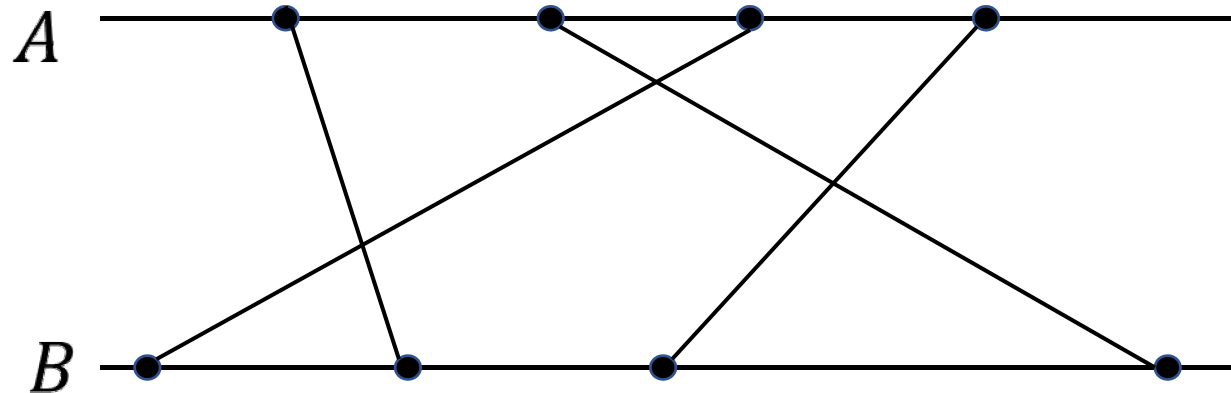
A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



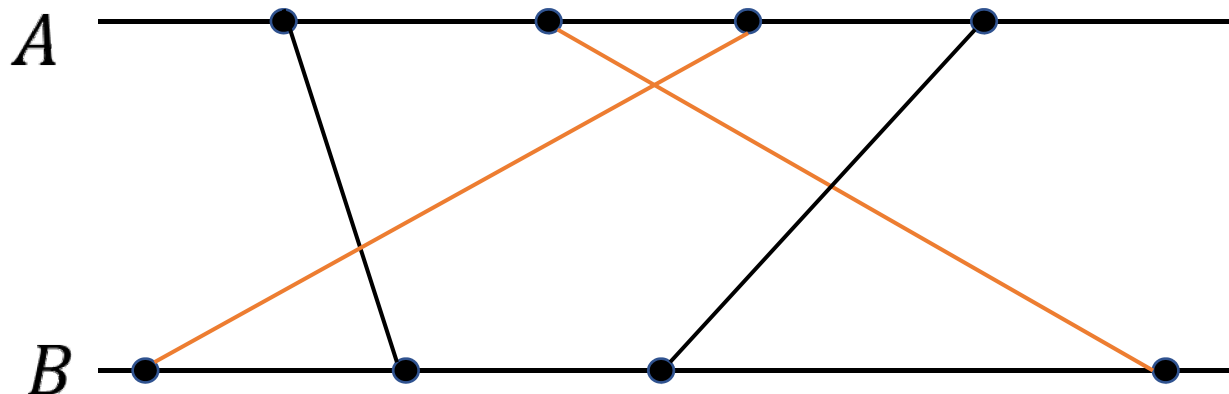
A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



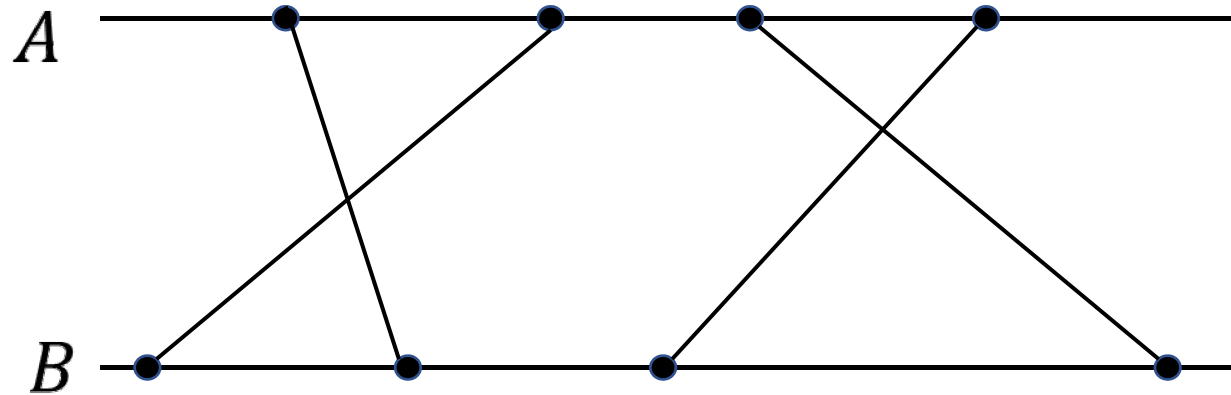
A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



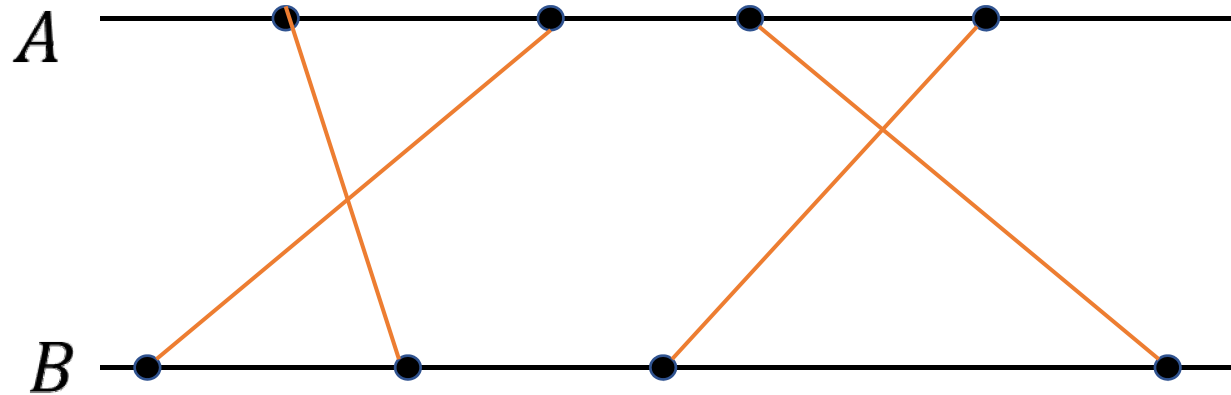
A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



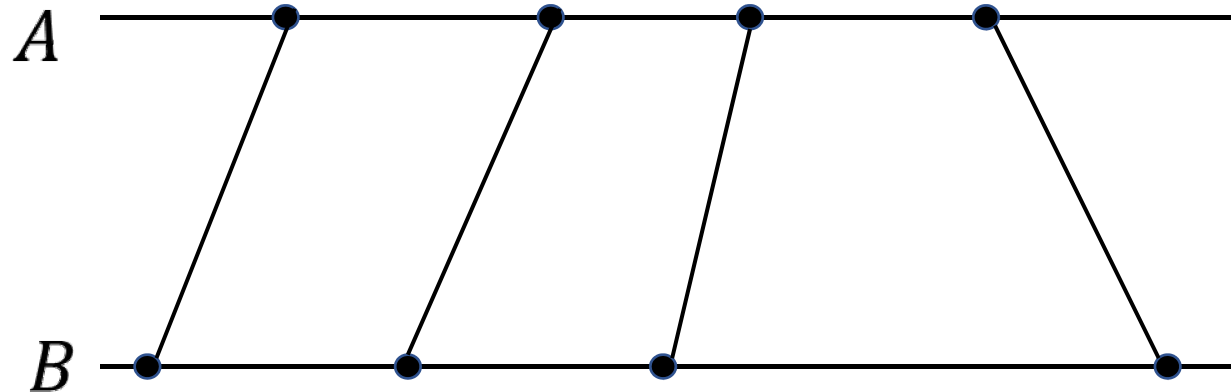
A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



A' の並べ方

- $A_1 \leq \dots \leq A_N$ 及び $B_1 \leq \dots \leq B_N$ があり、 A の並べ替え A' を動かす。 $A'_i - B_i$ の最大値が最小となるのは $A' = A$ のときである。



小課題 2

- 取り除く要素を固定 $\cdot \cdot \cdot$ N 通り
- A' の並べ方を固定 $\cdot \cdot \cdot$ 1 通り
- $\max\{A'_i - B_i, 0\}$ の最大値 $\cdot \cdot \cdot$ N 回計算
- 間に合う！

小課題 3

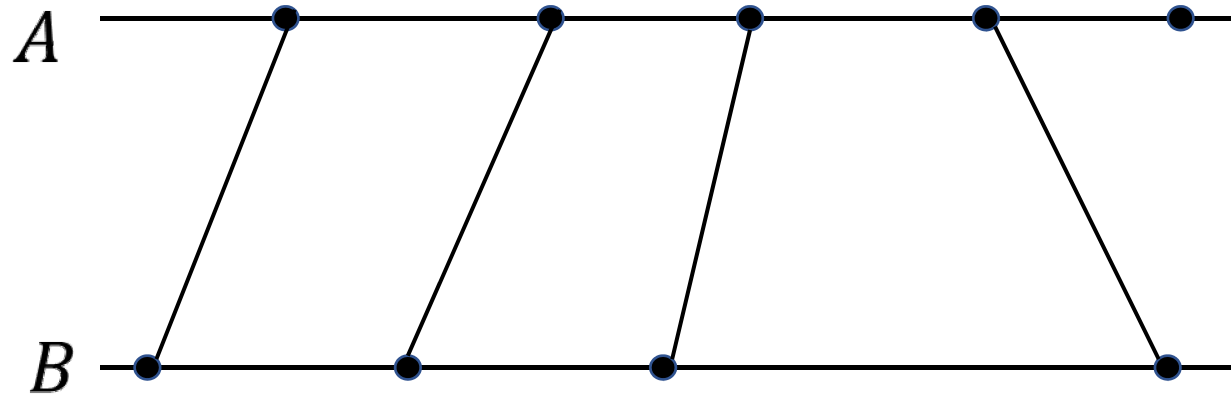
- $N \leq 200000$ なので計算回数減らす必要あり
- 取り除く要素を固定 $\cdot \cdot \cdot$ N 通り
- A' の並べ方を固定 $\cdot \cdot \cdot$ 1 通り
- $\max\{A'_i - B_i, 0\}$ の最大値 $\cdot \cdot \cdot$ N 回計算

小課題 3

- $N \leq 200000$ なので計算回数減らす必要あり
- 取り除く要素を固定 $\cdot \cdot \cdot N$ 通り
- A' の並べ方を固定 $\cdot \cdot \cdot 1$ 通り
- $\max\{A'_i - B_i, 0\}$ の最大値 $\cdot \cdot \cdot$ ~~N 回計算~~

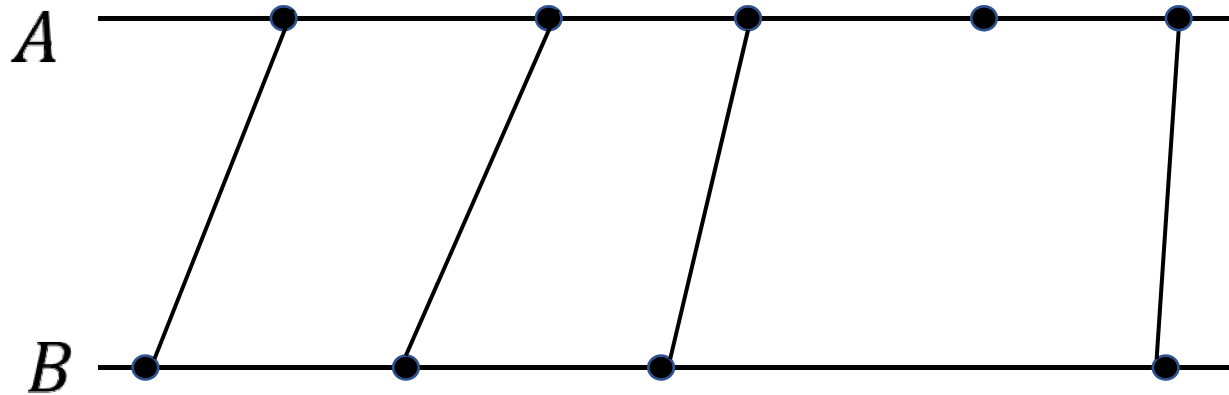
小課題 3

- 除く要素を動かしたときの変動をみる



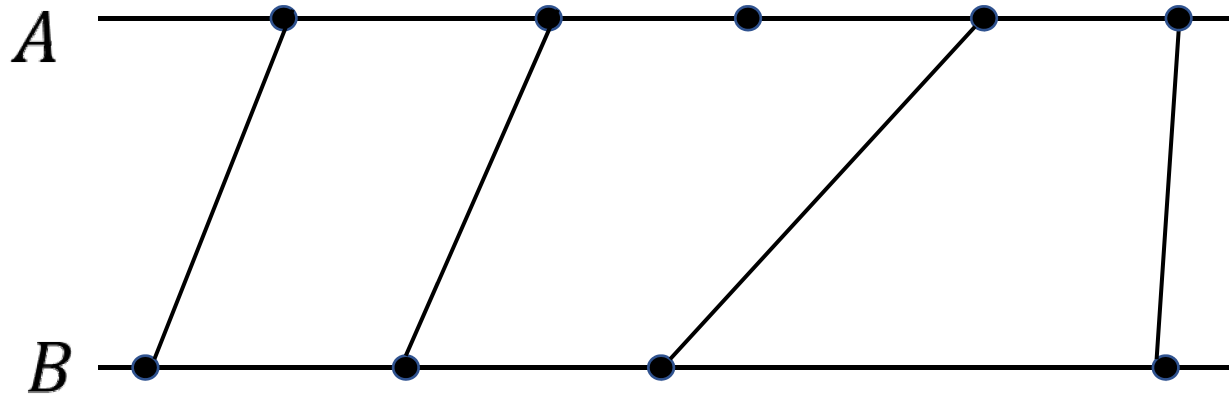
小課題 3

- 除く要素を動かしたときの変動をみる



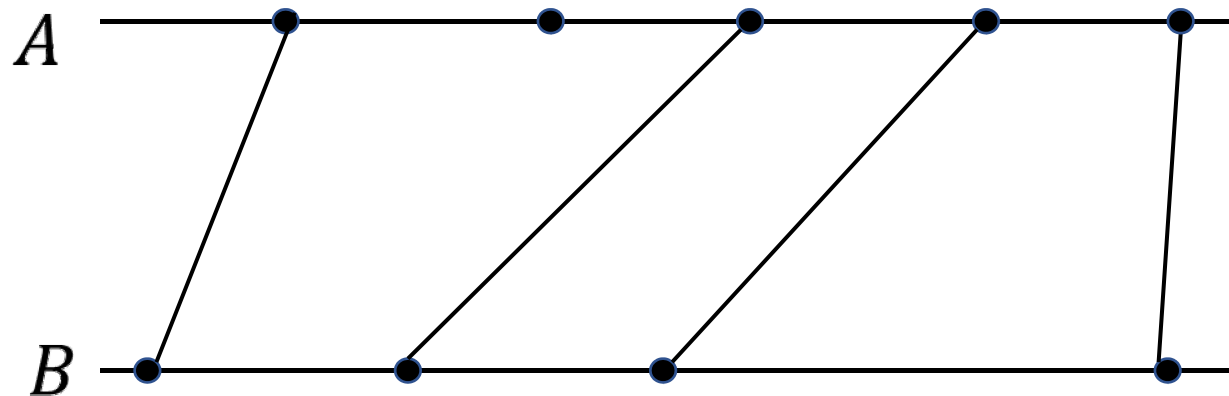
小課題 3

- 除く要素を動かしたときの変動をみる



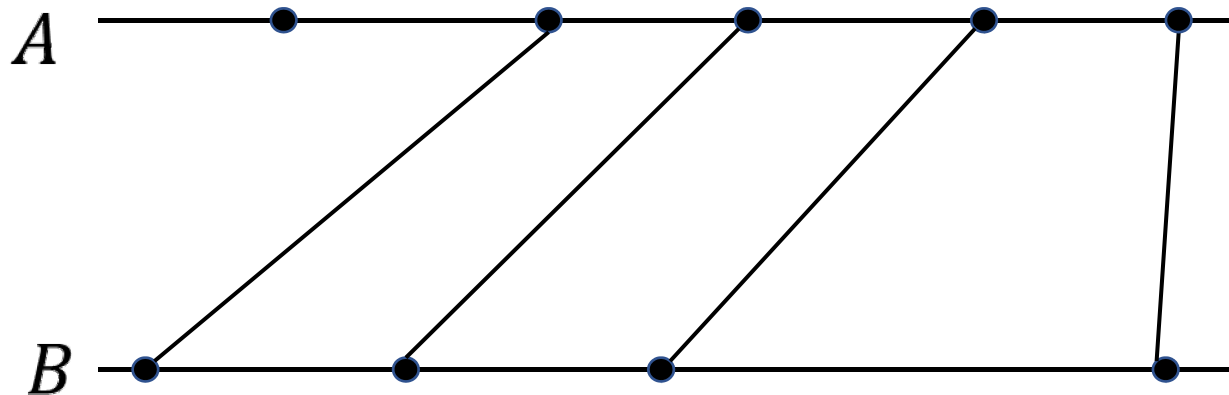
小課題 3

- 除く要素を動かしたときの変動をみる



小課題 3

- 除く要素を動かしたときの変動をみる



小課題 3

- A をソートした後は除く要素を一つずつずらし
ても $A'_i - B_i$ の集合は高々一つしか変化しない
– $N \log N$ の計算量で解ける

小課題 3

- A をソートした後は除く要素を一つずつずらし
ても $A'_i - B_i$ の集合は高々一つしか変化しない
– $N \log N$ の計算量で解ける
- そもそも $A'_i - B_i$ の最大値は単調増加
– ソートを除けば線形時間で解ける

J J O O I I 2

解説：Pulmn

問題概要

- J, O, I で構成された文字列 S が与えられる
- 次の 3 種類の操作が実行できる
 - 1 : 先頭の文字を削除
 - 2 : 末尾の文字を削除
 - 3 : 任意の文字を削除
- S を J, O, I の順に K 個並べてつなげた文字列 $JJ...JO...OI...II$ に変換するのに必要な 3 番目の操作回数の最小値は？

Sample (S="0JIJ0I0IIIJ" K=2)

• 0JIJ0I0IIIJ

• .JIJ0I0IIIJ

• .JIJ0I0II.

• .J.J0I0II.

• .J.J0.0II.

操作 1 : 先頭の文字を削除

操作 2 : 末尾の文字を削除

操作 3 : 任意の文字を削除

操作 3 : 任意の文字を削除

小課題 1 (N≤21)

- N が小さいので残す位置で全探索可能 (${}_NC_{3K}$ 通り)
- 残す位置を決めた時、操作 3 の最小回数は？

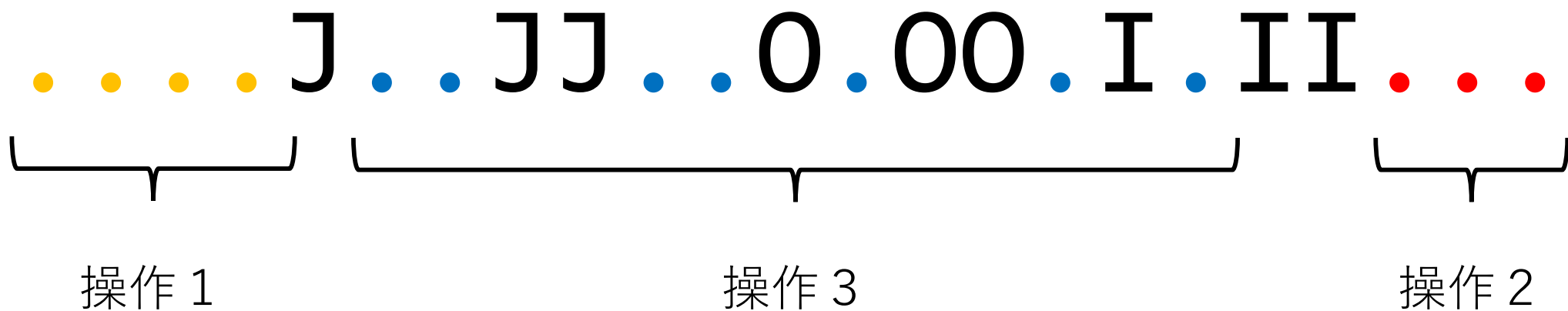
考察

- 残す位置を決めた時、操作 3 の最小回数は？

...J...JJ...0...00...I...II...

考察

- 残す位置を決めた時、操作 3 の最小回数は？



- 操作 3 を行う回数は $O(N)$ で計算可能

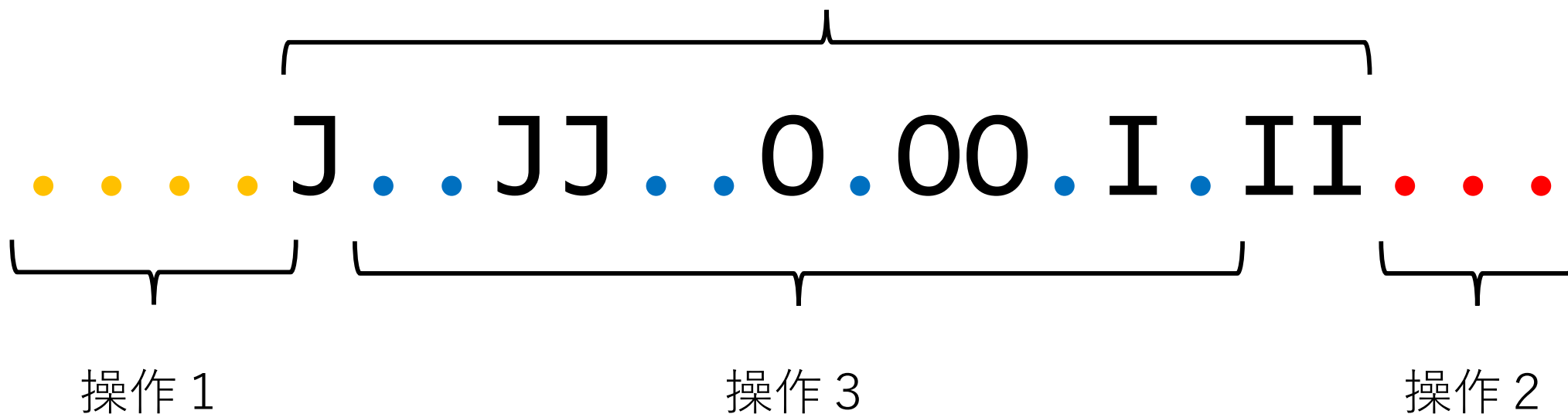
小課題 1 (N≤21)

- 全列挙するのに $O(2^N)$ かかるので、全体で $O(N2^N)$
- 1点獲得

小課題 2 (N≤3000)

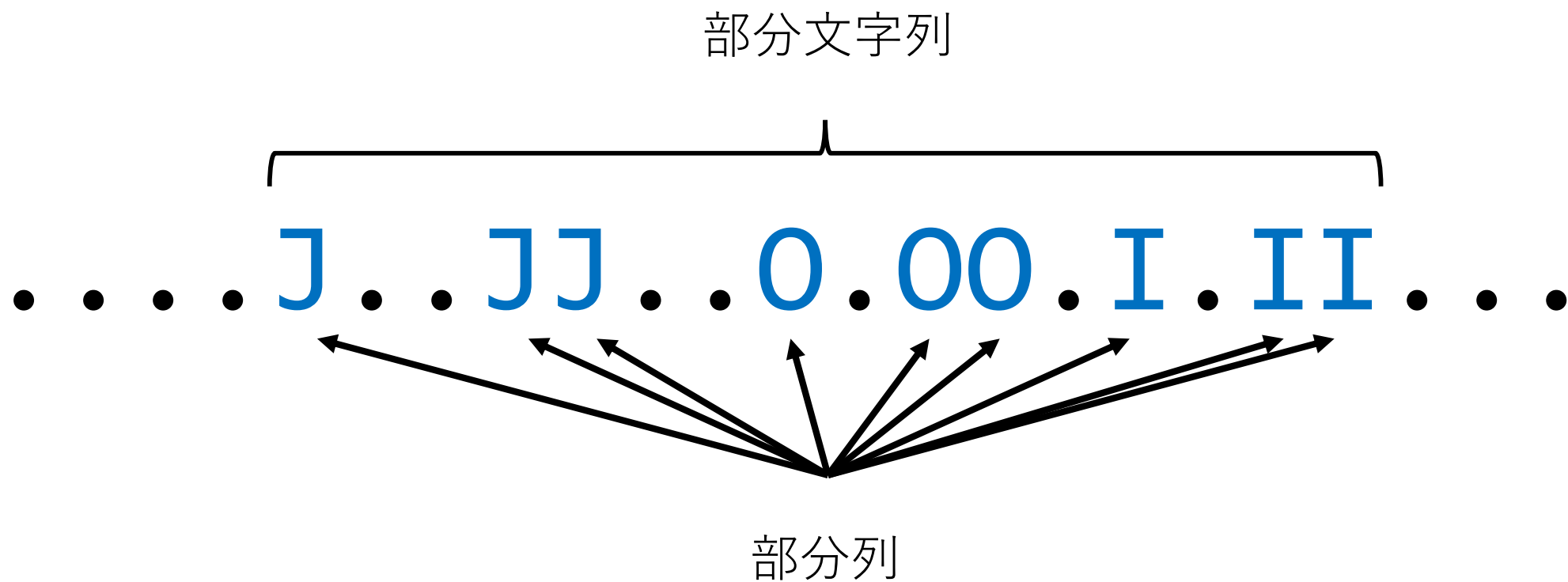
- N が大きいので全探索はできない

(操作 3 の回数) + $3K$ ← これを最小化する



考察

- レベルKのJOI文字列を部分列として含む部分文字列の長さの最小値を求める問題に帰着できる



考察

- 左端を固定してみる

JJJ000III

0I0I|J0IJJJIOJ00JI0IIJ0J

考察

- 左端を固定してみる

0101J01JJJI0J00J1011J0J

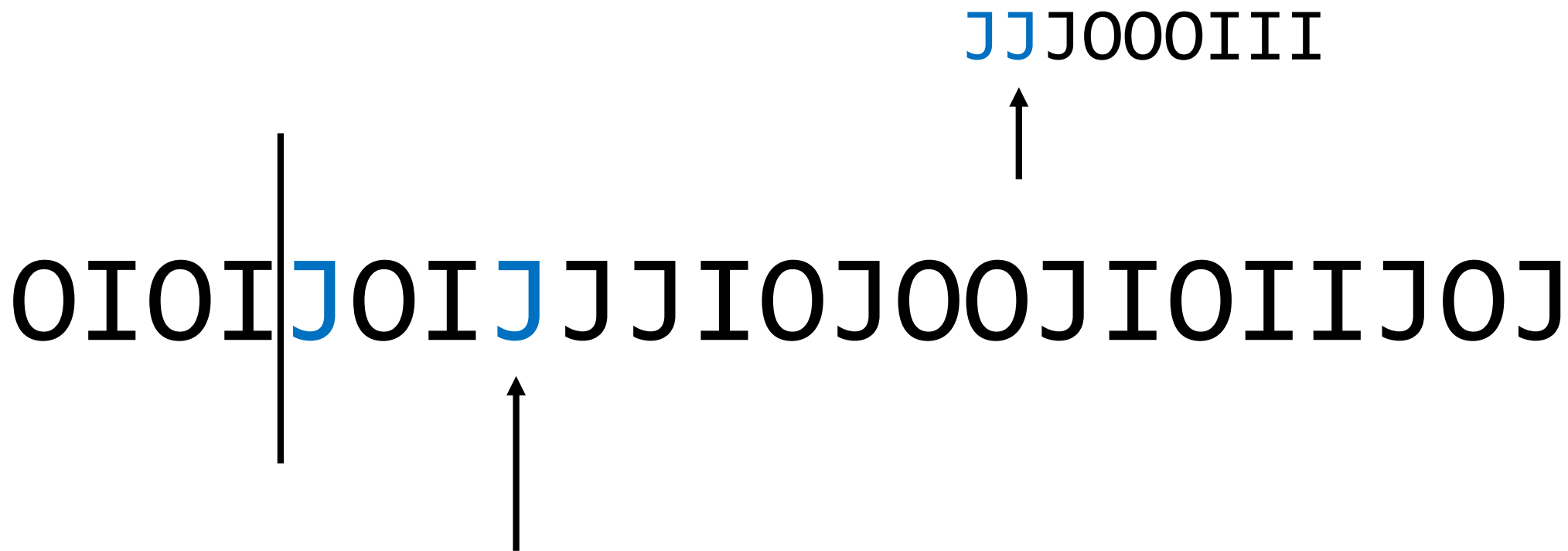
↑

JJJ000III

↑

考察

- 左端を固定してみる



考察

- 左端を固定してみる

0101|J0IJJJIOJ00JIOIJJ0J

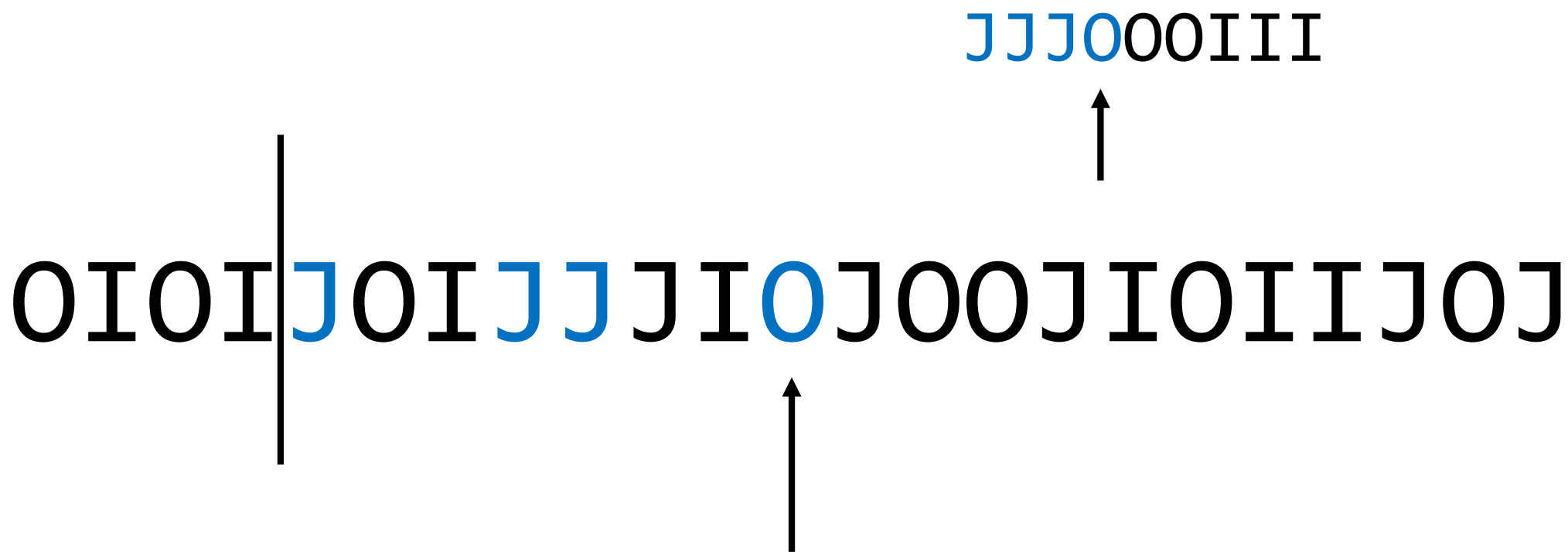
JJJ000III

↑

↑

考察

- 左端を固定してみる



考察

- 左端を固定してみる

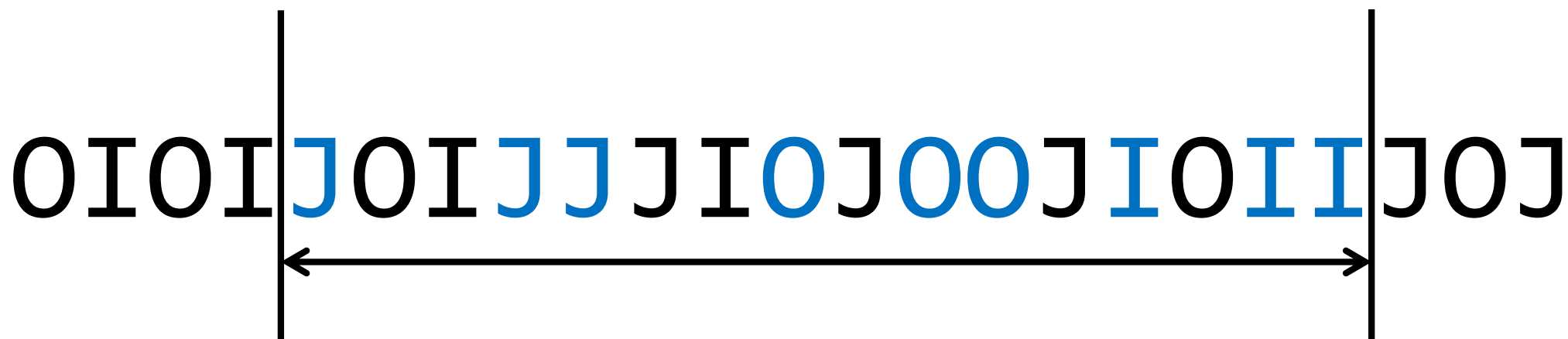
0101|J0IJJJIOJ00JIOIJJ0J

JJJ000III

The diagram illustrates a sequence of characters: 0101|J0IJJJIOJ00JIOIJJ0J. A vertical line is positioned between the fourth character (1) and the fifth character (J). Above the sequence, the string JJJ000III is shown in blue, with an arrow pointing to the eighth character (I) of the main sequence. Below the sequence, an arrow points to the tenth character (I).

考察

- 左端を固定してみる



考察

- 左端を固定したときのレベル K のJOI文字列を部分列として含む部分文字列の長さの最小値が $O(N)$ で計算できた
- 左端は N 通りしかないので、全体の計算量は $O(N^2)$
- 13点獲得

小課題 3 (N≤200000)

- $O(N^2)$ では遅い
- レベル K の JOI 文字列の特徴をうまく利用して右端の計算を高速化できる？

考察

- レベル K のJOI文字列の特徴をうまく利用して右端の計算を高速化できる？



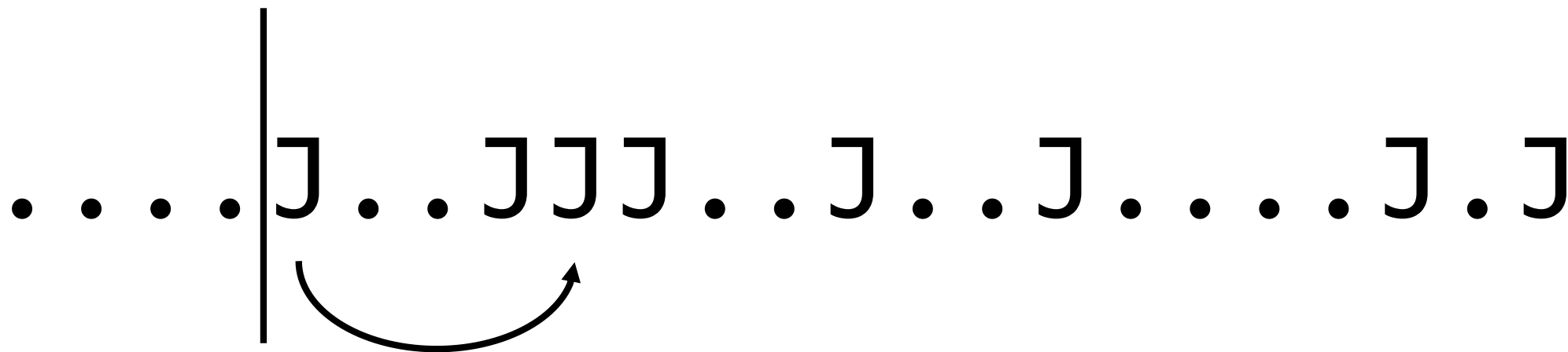
考察

- 左端を固定したとき各文字の K 個目の位置が既にわかっていると $O(1)$ で計算できる



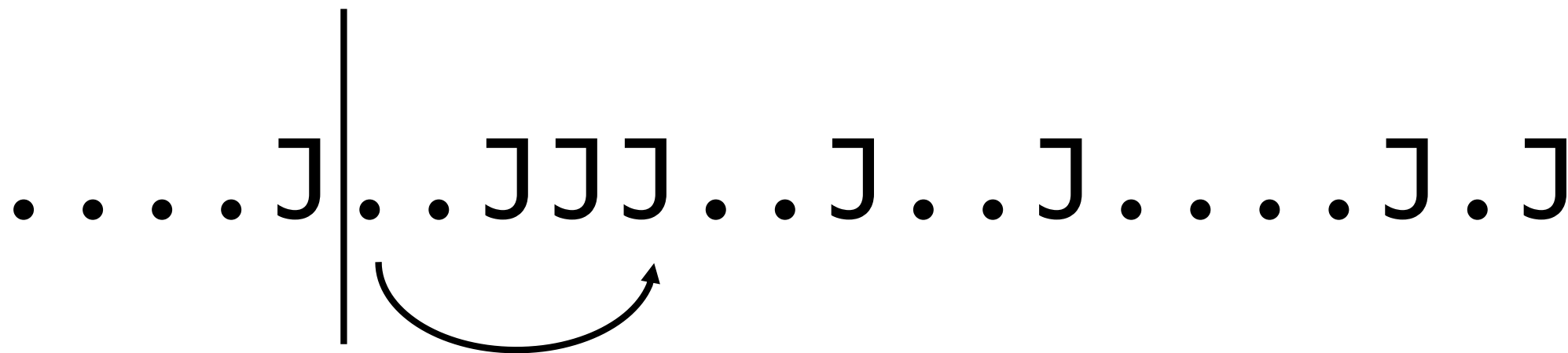
考察

- 左端を固定したとき各文字の K 個目の位置は尺取り法を用いると全体で $O(N)$ で計算可能



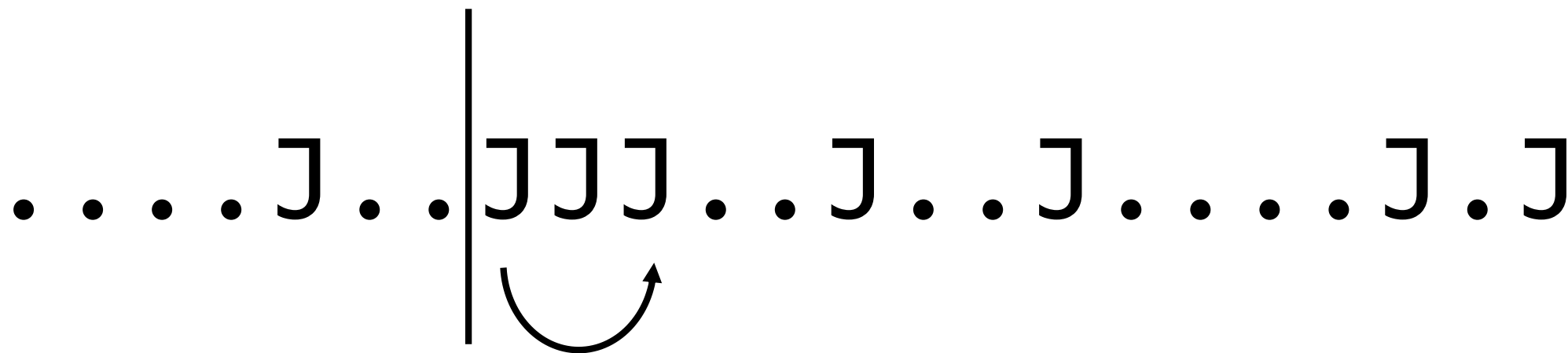
考察

- 左端を固定したとき各文字の K 個目の位置は尺取り法を用いると全体で $O(N)$ で計算可能



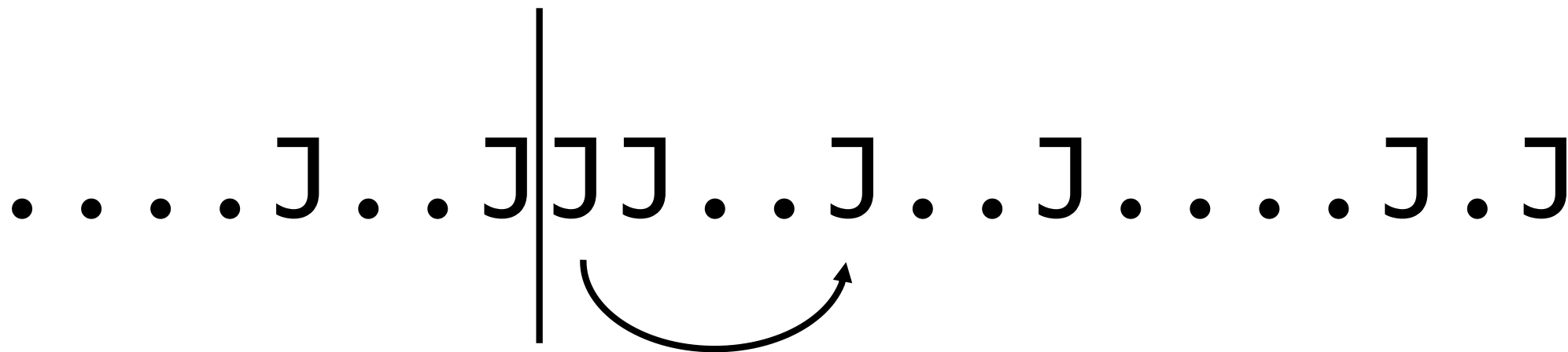
考察

- 左端を固定したとき各文字の K 個目の位置は尺取り法を用いると全体で $O(N)$ で計算可能



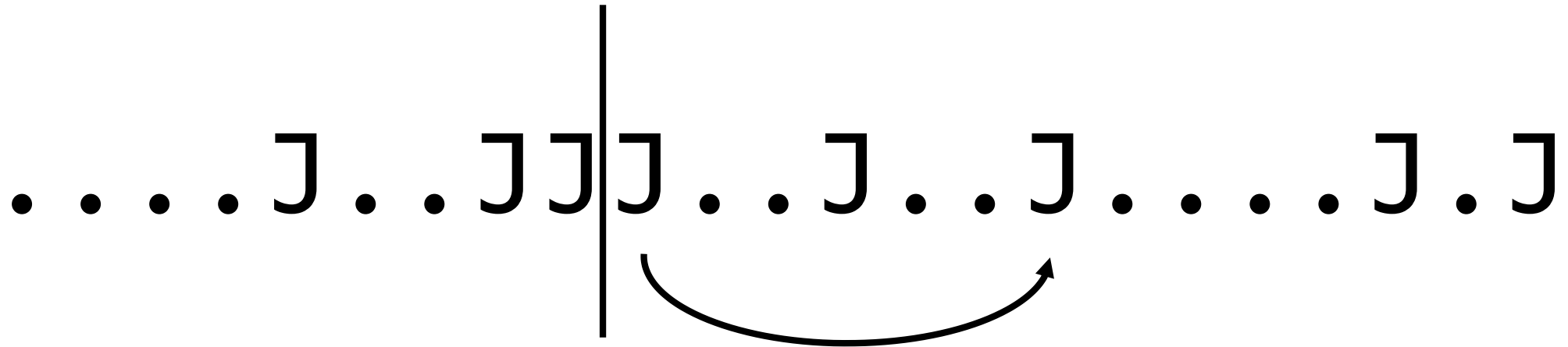
考察

- 左端を固定したとき各文字の K 個目の位置は尺取り法を用いると全体で $O(N)$ で計算可能



考察

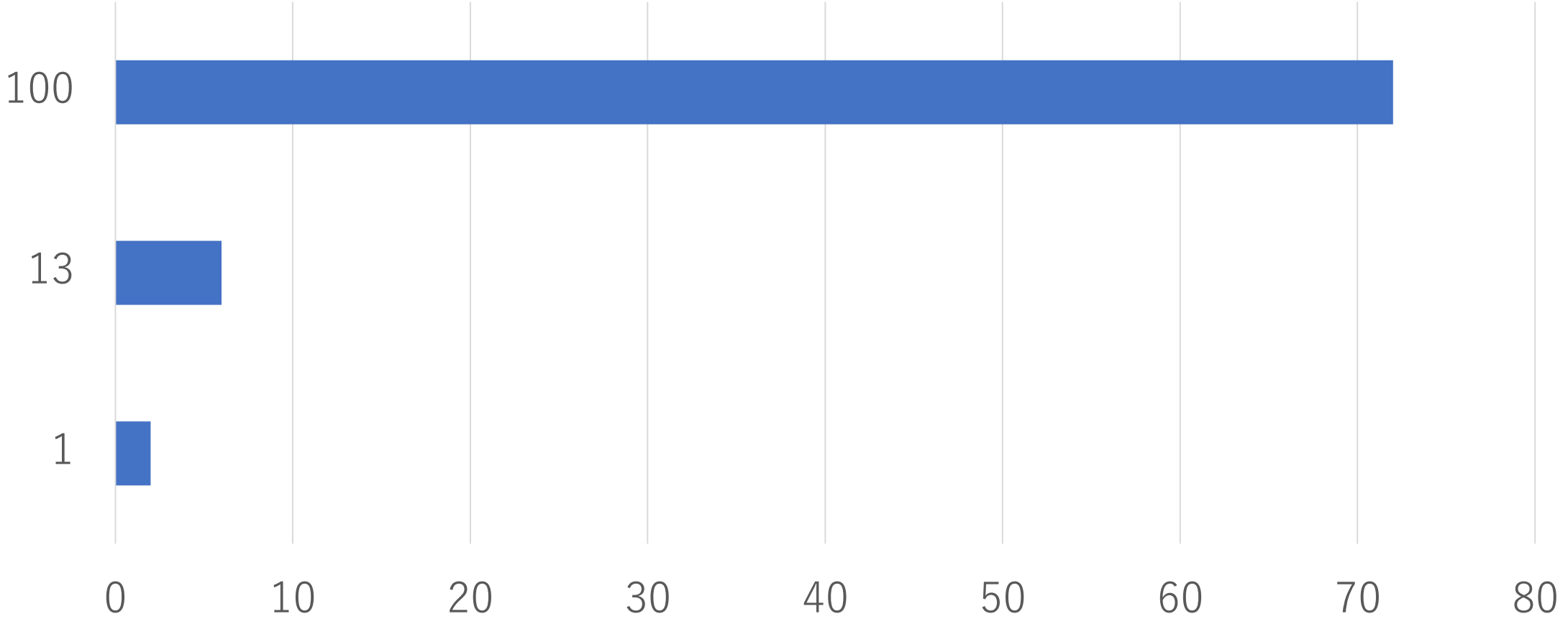
- 左端を固定したとき各文字の K 個目の位置は尺取り法を用いると全体で $O(N)$ で計算可能



小課題 3 ($N \leq 200000$)

- 左端を固定したとき各文字の K 個目の位置は尺取り法を用いると全体で $O(N)$ で計算可能
- 左端を固定したときのレベル K を部分列として含む部分文字列の長さの最小値が $O(1)$ で計算可能
- 全体で $O(N)$
- 100点獲得

得点分布



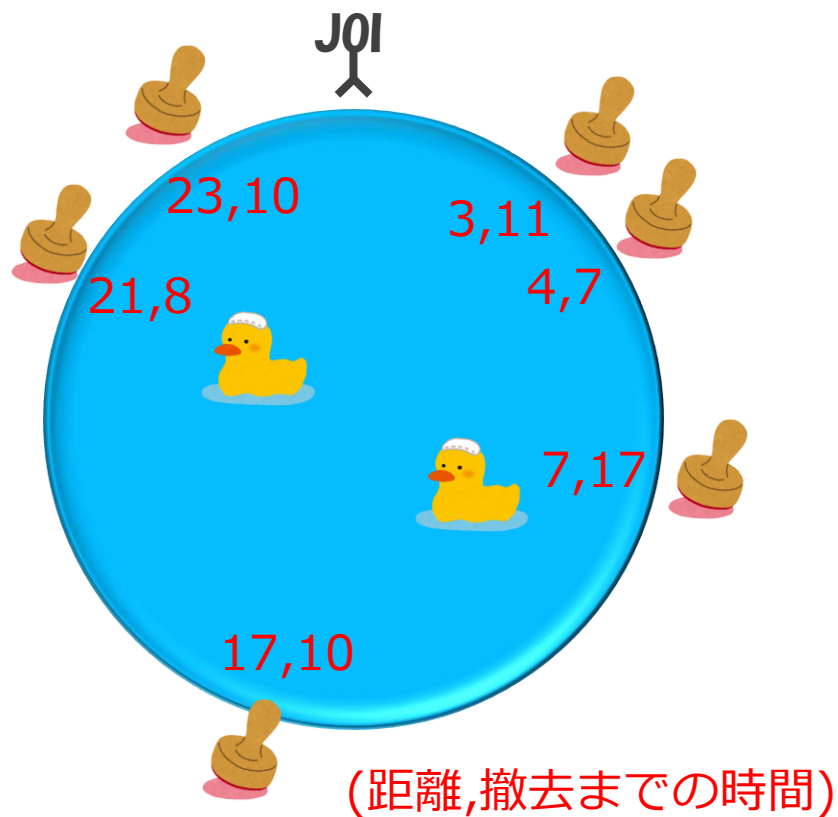
問題 3 スタンプラリー 3 (Collecting Stamps 3)

解説：細川 寛晃

問題概要

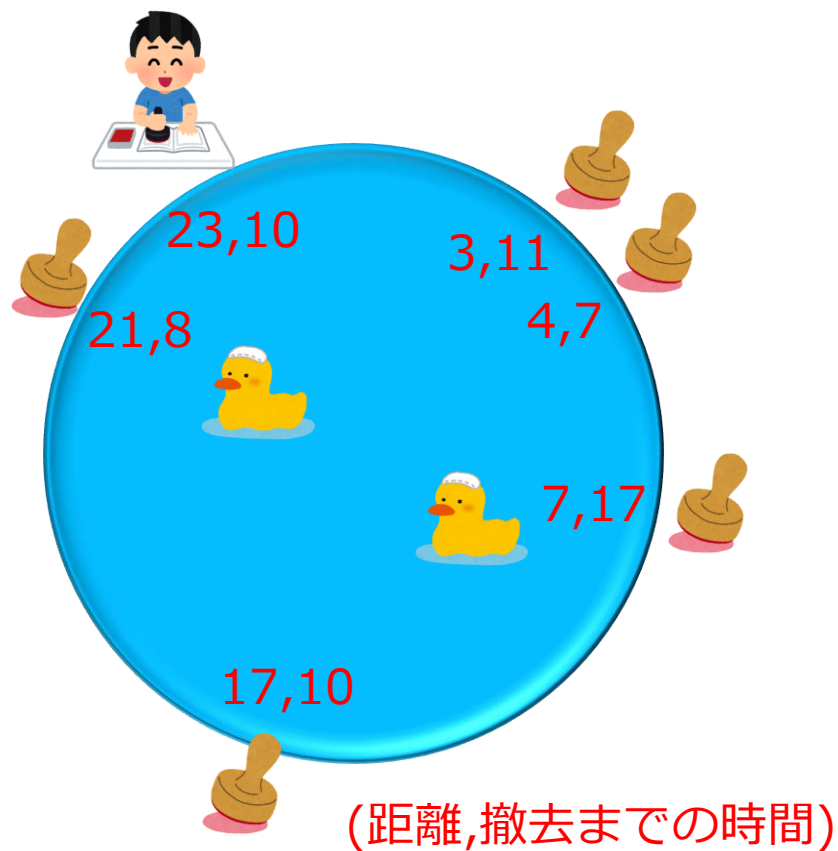
- スタンブラリーをする
- 環状にスタンプ台が並んでいる
- スタンブラリー台はそれぞれ定められた時間経つと押せなくなる
- 最大でいくつのスタンプを押せるか？

入出力例1



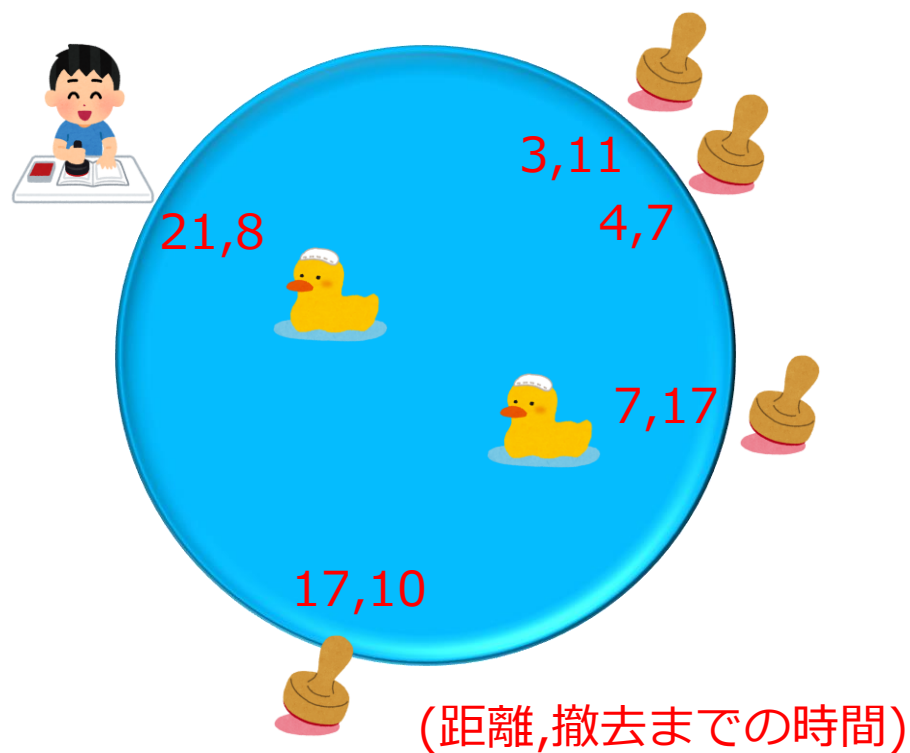
- $N = 6, L = 25$
- 経過時間: 0
- 押したスタンプ数: 0
- 左へ回る

入出力例1



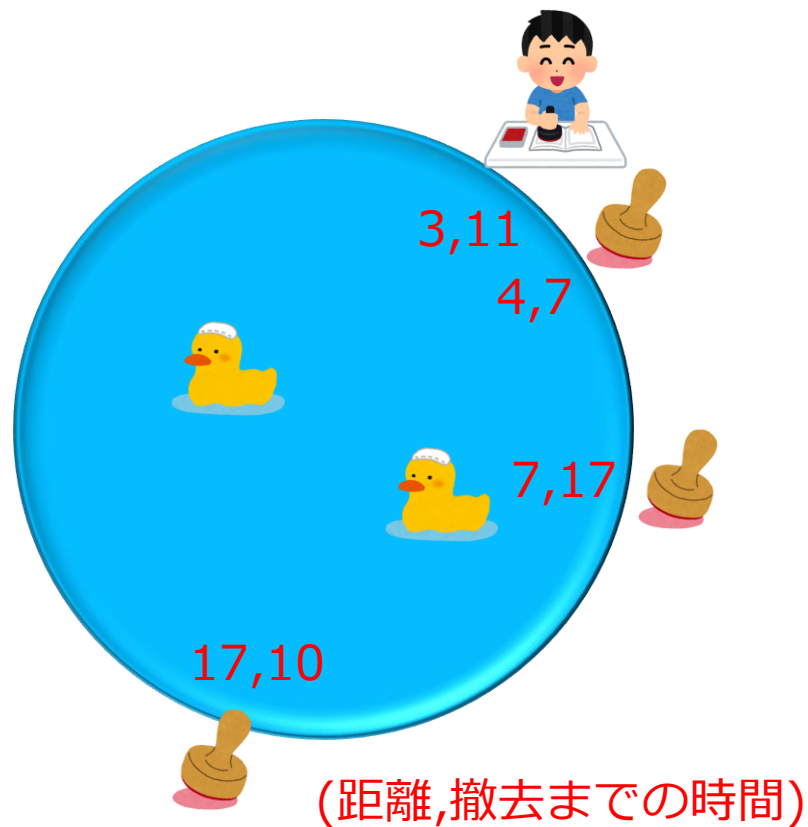
- $N = 6, L = 25$
- 経過時間: 2
- 押したスタンプ数: 1
- 左へ回る

入出力例1



- $N = 6, L = 25$
- 経過時間: 4
- 押したスタンプ数: 2
- 右へ回る

入出力例1



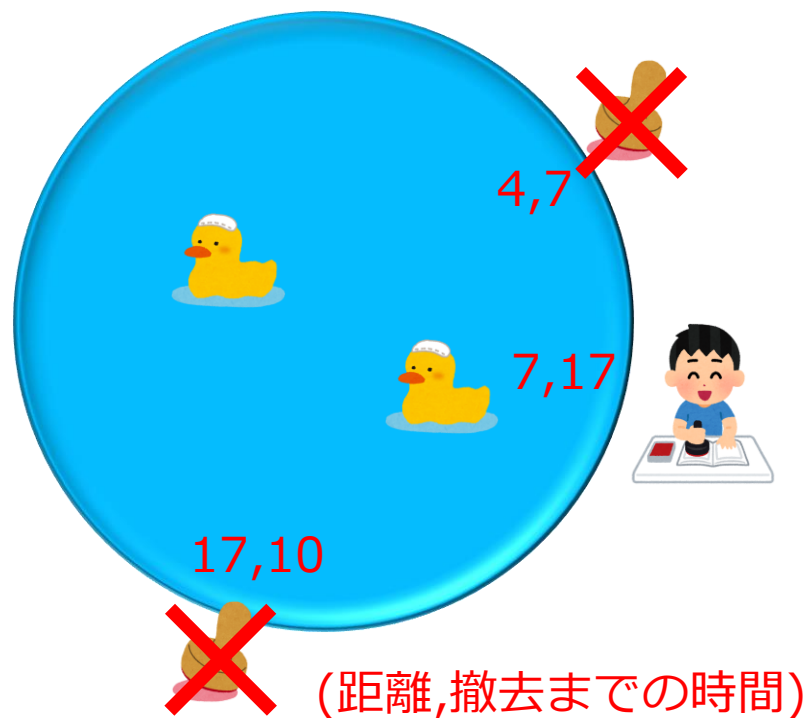
- $N = 6, L = 25$
- 経過時間: 11
- 押したスタンプ数: 3
- 右へ回る

入出力例1



- $N = 6, L = 25$
- 経過時間: 12
- 押したスタンプ数: 3
- 右へ回る

入出力例1



- $N = 6, L = 25$
- 経過時間: 15
- 押したスタンプ数: 4
- これが最適

小課題1(5点)

- $N \leq 12$
- $L, T_i \leq 200$

とりあえず全列挙

- スタンプ台を訪れる順番を全列挙
- 各スタンプ台でスタンプを押せるかどうか判定
- 計算量 $O(N \times N!)$
- 小課題1が……？
- 解けない(悲しい)
- ($N = 12$ のとき $N \times N! = 5,748,019,200 \doteq 5.7 * 10^9$)
- 枝刈りをすれば間に合う可能性あり

動的計画法を考える

- 訪れたスタンプ台、今いるスタンプ台、経過時間が同じならそれ以降の動きに影響を与えない

→動的計画法(DP)

解法1

- $dp[S][i][t] :=$ 訪れたスタンプ台の集合が S 、今いるスタンプ台が i 、経過時間が t のときの押したスタンプ数の最大値
 - S は N bitの整数で表せばよい
 - いわゆるbit DP
- 遷移は訪れていないスタンプ台へ移動
- 状態数 $O(2^N NT)$ 、遷移 $O(N)$ なので計算量は $O(2^N N^2 T)$
- 小課題1が……解ける！(嬉しい)
- ($N = 12$ 、 $T = 200$ のとき $2^N N^2 T = 117,964,800 \doteq 1.2 * 10^8$)
- 5点獲得

運命の分かれ道

- DPの状態数をなんとかしたい

$$dp[\underbrace{S}_{O(2^N N)}][\underbrace{i}_{O(T)}][t]$$

こっちをなんとかする！
小課題3へ

こっちをなんとかする！
小課題2へ

小課題2(10点)

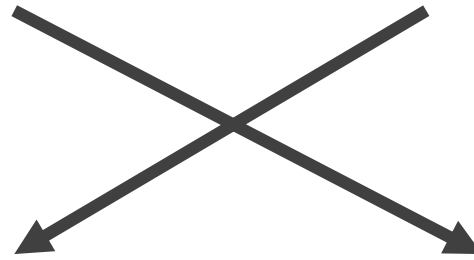
- $N \leq 15$
- $L, T_i \leq 10^9$

添字と値

- $dp[S][i][t]$:= 訪れたスタンプ台の集合が S 、今いるスタンプ台が i 、経過時間が t のときの押したスタンプ数の最大値
- 添字の t は 10^9 までである可能性がある
- 一方 $dp[S][i][t]$ の値は高々 $N \leq 200$
 - スタンプは N 個しかないので
- 値の小ささを生かせないか？

添字と値の入れ替え

$dp[S][i][t] := \text{経過時間が } t \text{ のときの } \underline{\text{スタンプ数}} \text{ の } \text{最大値}$



$dp[S][i][k] := \text{スタンプ数が } k \text{ のときの } \underline{\text{経過時間}} \text{ の } \text{最小値}$

添字と値の入れ替え

- 注: 入れ替え後は最小値を求めることになります
 - 最適を求めたい
 - スタンプ数が同じなら経過時間が短いほうが良い
- DPの値が小さい(種類数が少ない)時に使える典型テク

解法2

- $dp[S][i][k] :=$ 訪れたスタンプ台の集合が S 、今いるスタンプ台が i 、押したスタンプ数が k のときの経過時間の最小値
- 遷移は未訪問のスタンプ台への移動
- 遷移時、 k に1を足す(スタンプを押せる)かどうかはDPの値を見て判定
- 到達できる(値がINFではない)状態の中で k が最大のものが答え
- 状態数 $O(2^N N^2)$ 、遷移 $O(N)$ なので計算量 $O(2^N N^3)$
- 小課題1と小課題2が解けて15点

ちょっとした改善

- $dp[S][i] :=$ 押したスタンプ台の集合が S 、今いるスタンプ台が i のときの経過時間の最小値
- としてもよい
- 現在の押した個数は S の要素の数(立っているビットの数)
- 状態数が $O(2^N N)$ になるため、計算量は $O(2^N N^2)$
- 同じく15点

運命の分かれ道

- DPの状態数をなんとかしたい

$$dp[\underbrace{S}_{O(2^N N)}][\underbrace{i}_{O(T)}][t]$$

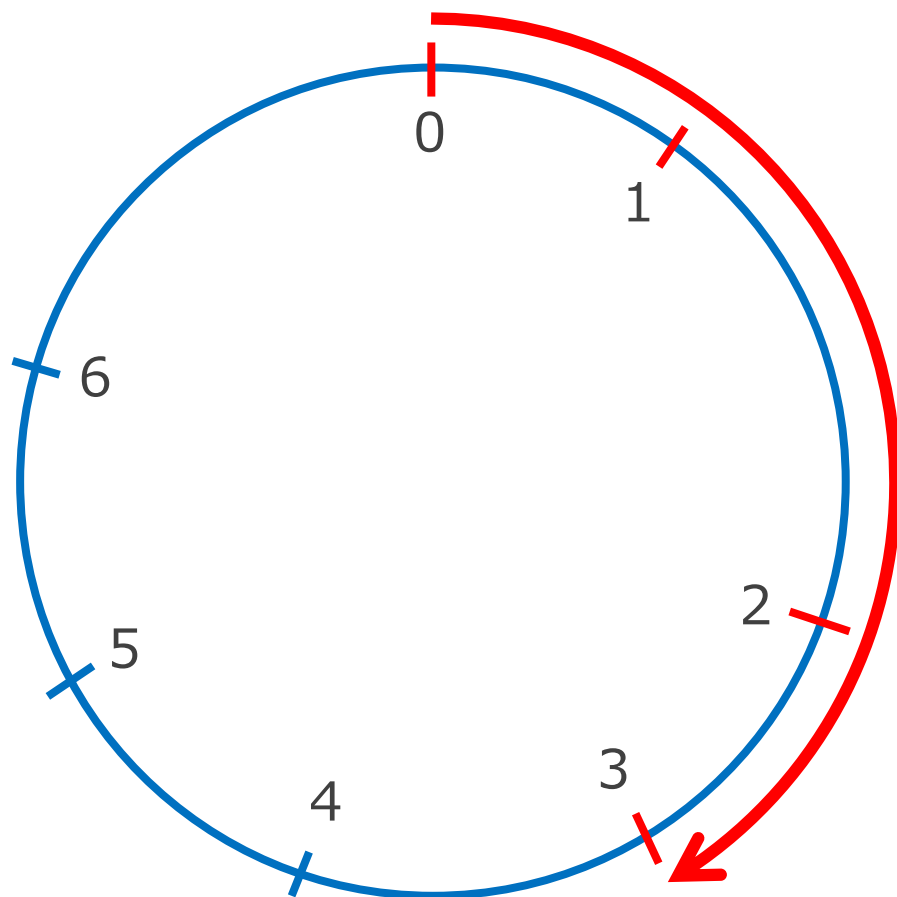
こっちをなんとかする！
小課題3へ

こっちをなんとかする！
小課題2へ

小課題3(10点)

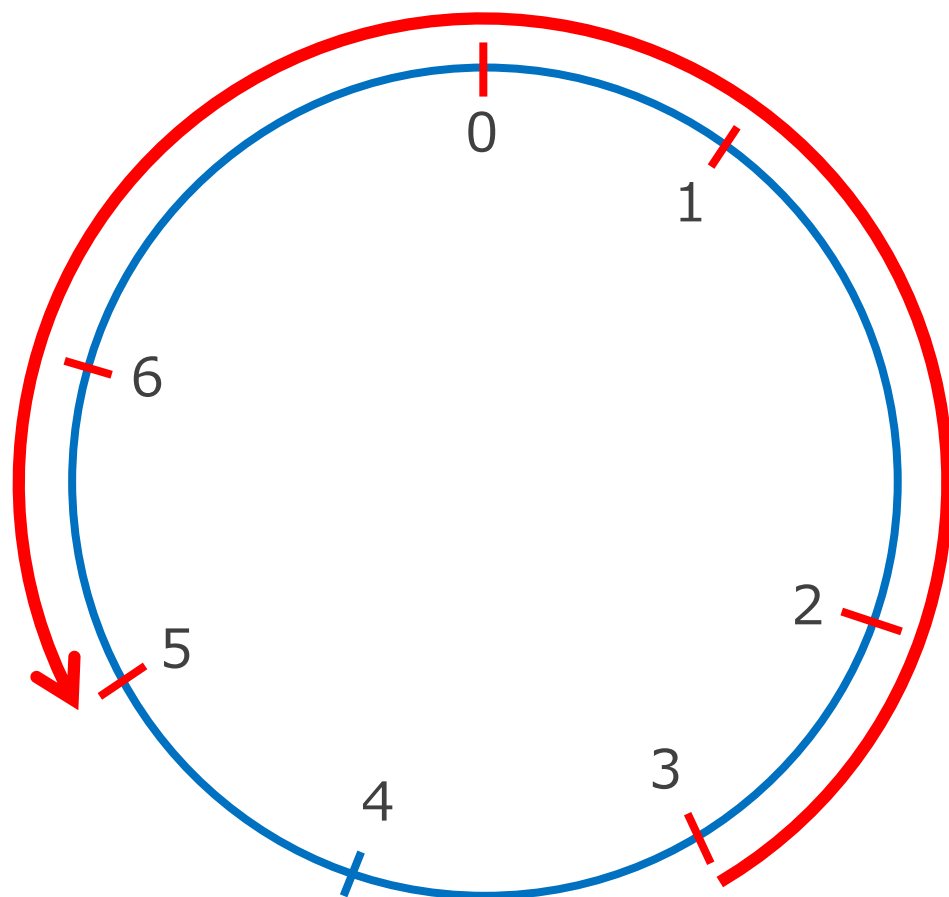
- $N \leq 200$
- $L, T_i \leq 200$
- 訪れたスタンプ台の情報の持ち方を少し考察してみる
- 以下、便宜上スタート地点をスタンプ台0とします
 - 実装時にもこう考えるとちょっと楽

訪れたスタンプ台って？



- 台0→台3へ移動
- 訪れた台は？
- ~~台0と台3のみ~~
- 台0,1,2,3
- 台0～3の区間！

訪れたスタンプ台って？



- さらに台3→台5へ移動
- 訪れた台は？
- 台5,6,0,1,2,3
- 台5～3の区間

訪れたスタンプ台って？

- 常に訪れたスタンプ台は区間になっている！！

→区間の両端を状態として持てばよさそう

解法3

- $dp_L[i][j][t]$:= 訪れた台の区間の左端が i 、右端が j 、時刻が t で
今左端にいるときの押したスタンプ数の最大値
- $dp_R[i][j][t]$:= 訪れた台の区間の左端が i 、右端が j 、時刻が t で
今右端にいるときの押したスタンプ数の最大値
- 遷移は区間の左端あるいは右端を1つ伸ばす
- 状態数 $O(N^2T)$ 、遷移 $O(1)$ なので計算量 $O(N^2T)$
- 小課題1と小課題3が解けて15点

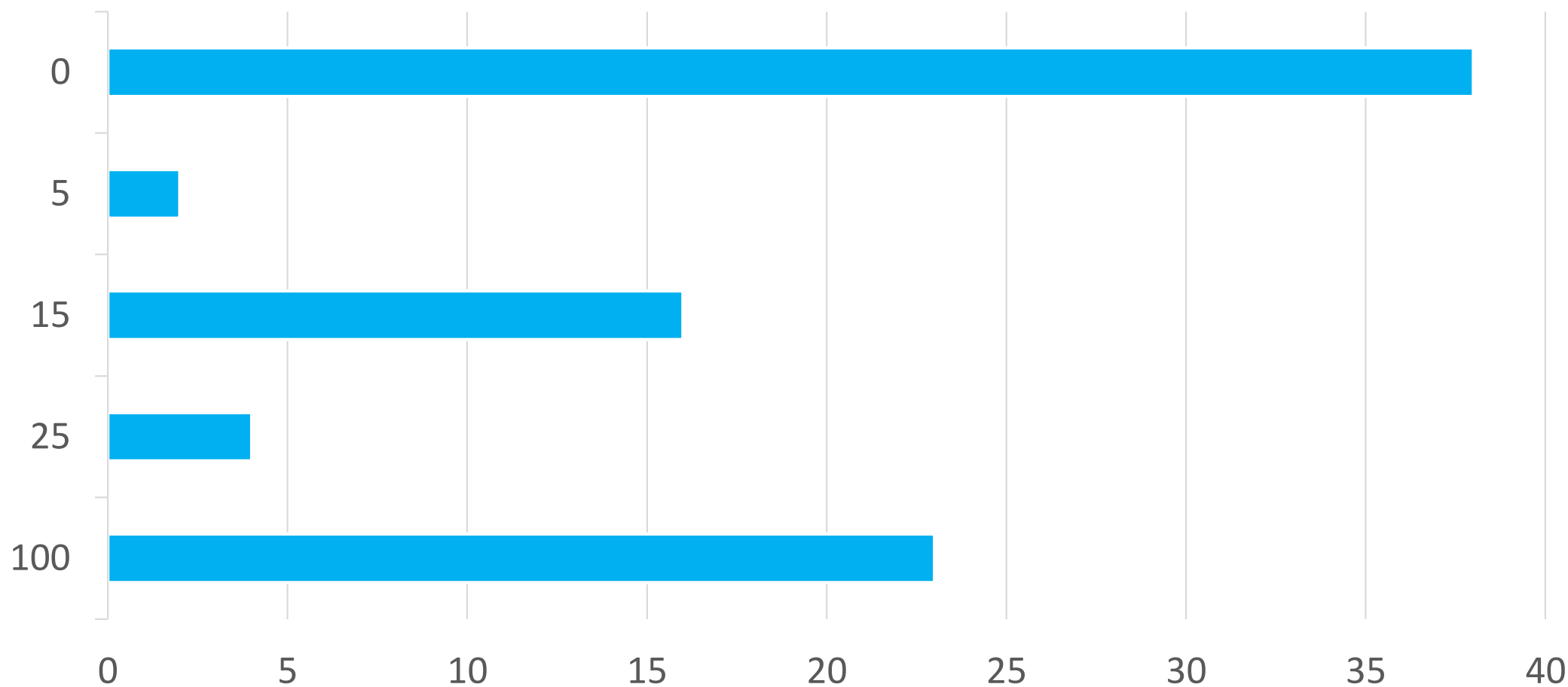
小課題4(75点)

- $N \leq 200$
- $L, T_i \leq 10^9$
- どちらもなんとかしたい
- 小課題2と小課題3の解法を融合させればよい
 - 区間にする+添字と値の入れ替え

解法4

- $dp_L[i][j][k]$:= 訪れた区間の左端が i 、右端が j 、押したスタンプ数が k で、今左端にいるときの経過時間の最小値
- dp_R も同様に定義
- 遷移は区間の左端あるいは右端を1つ伸ばす
- 状態数 $O(N^3)$ 、遷移 $O(1)$ なので計算量 $O(N^3)$
- 全ての小課題が解けて、満点！！！！

得点分布



問4 オリンピックバス

坂部 圭哉

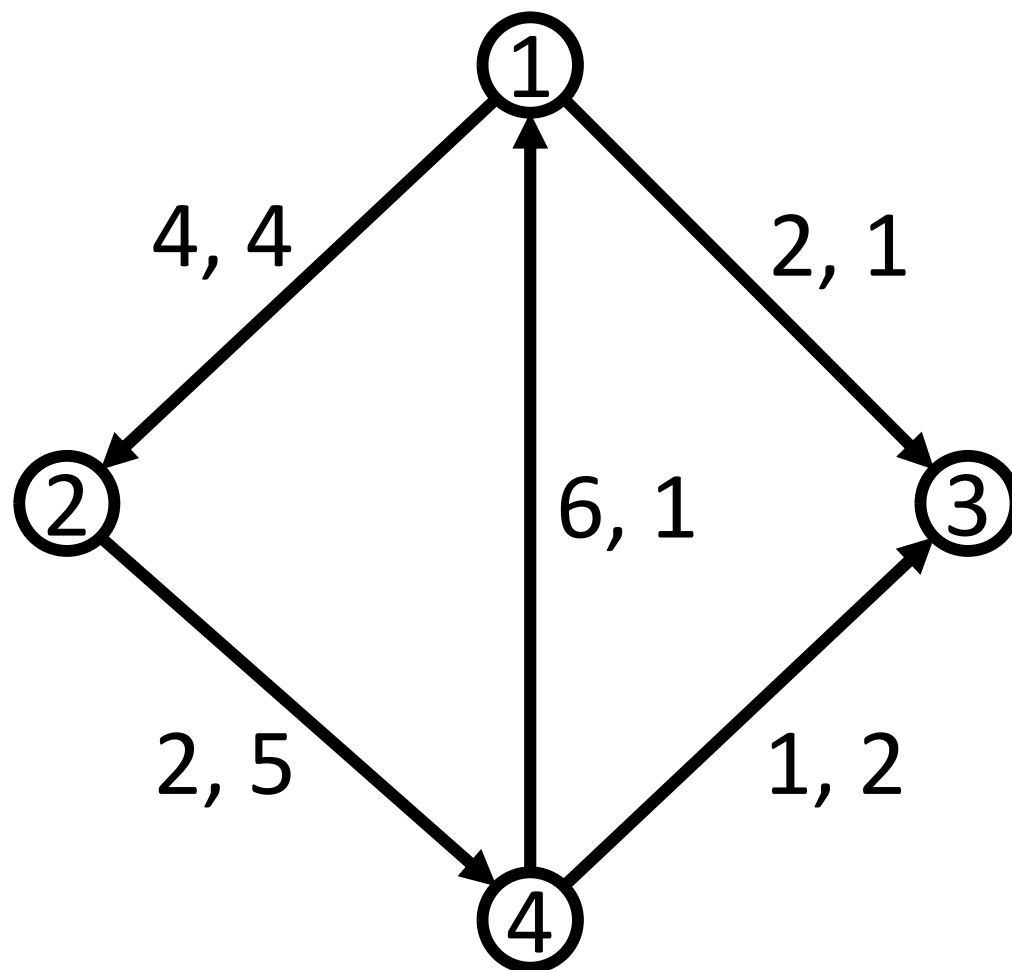
問題概要

- 有向グラフがある。
- 各辺には、移動コスト C と反転コスト D が定まっている。
- 辺を0本または1本反転させることができる。
- 次の3つの和を最小化したい。
 - 頂点1から頂点 N までの移動にかかるコスト
 - 頂点 N から頂点1までの移動にかかるコスト
 - 反転コスト

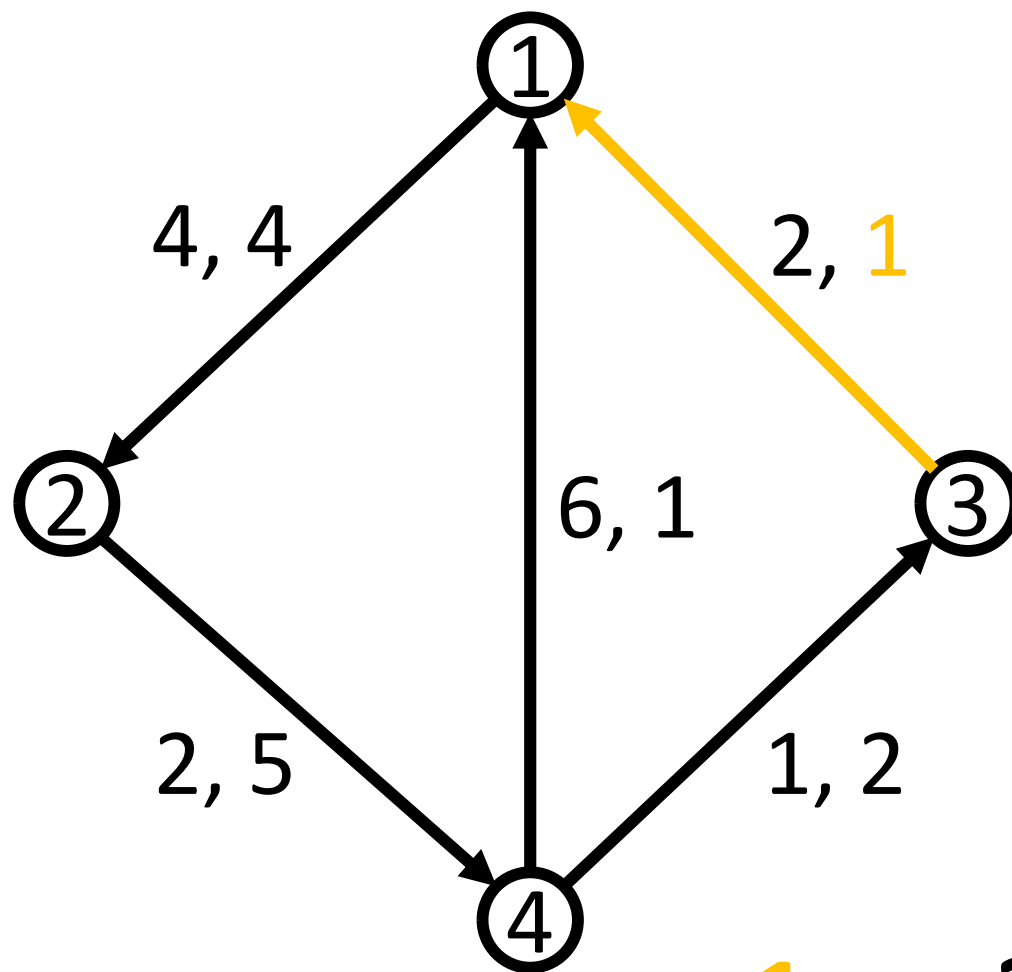
制約

- $N \leq 200$
- $M \leq 50,000$
→ 密な(辺の多い)グラフ

例

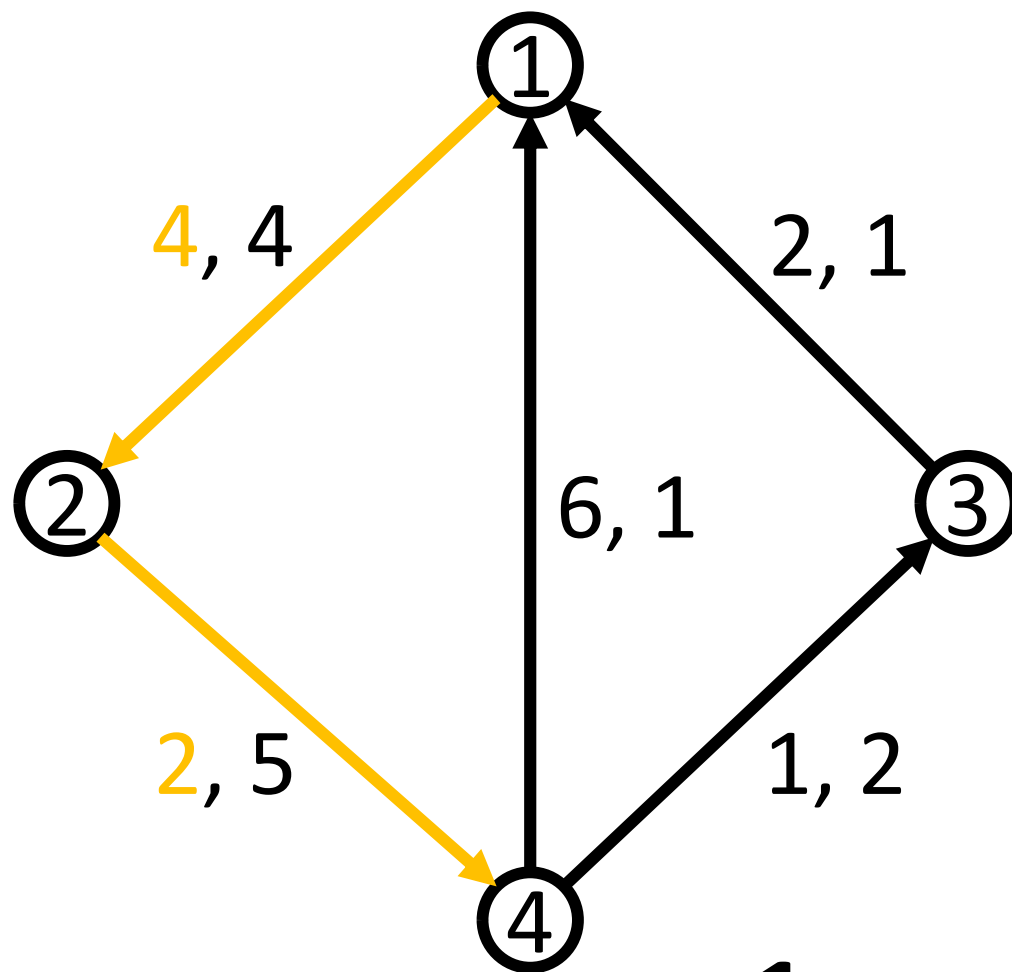


例



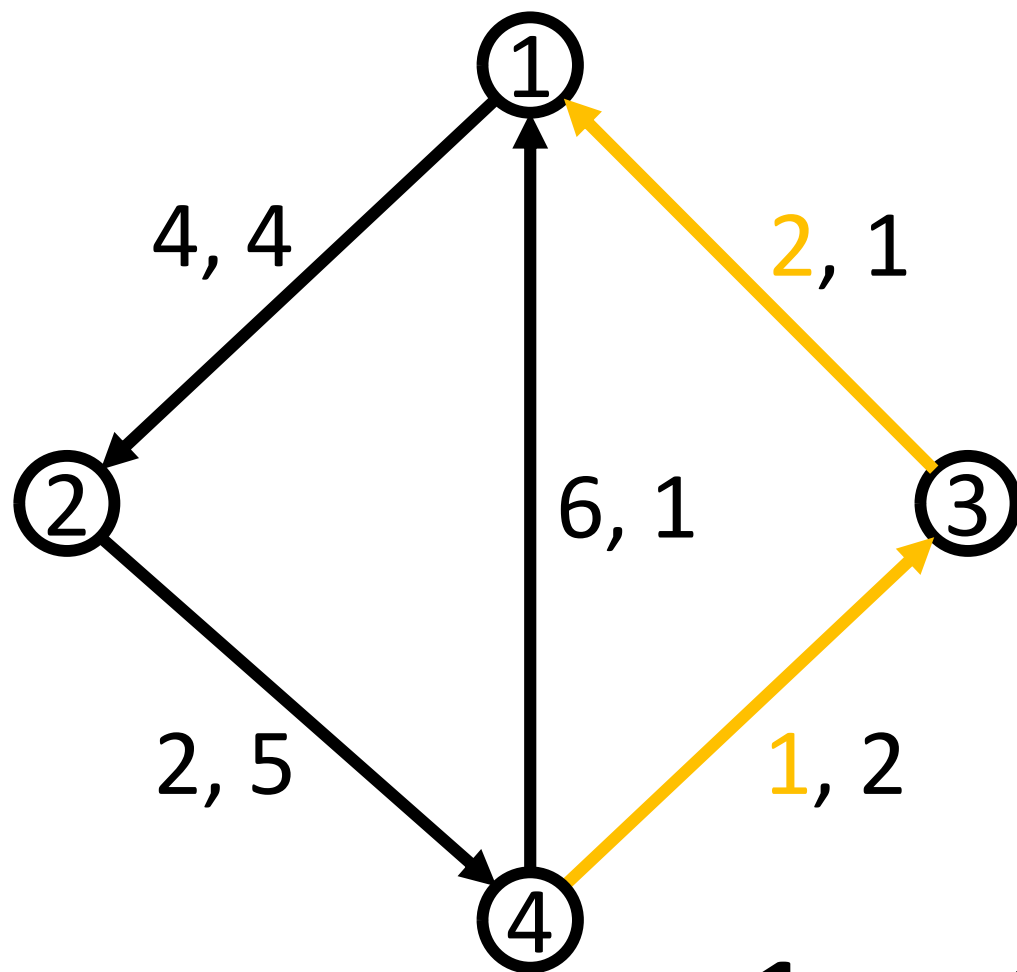
$$1 + ? + ? = ??$$

例



$$1 + 6 + ? = ??$$

例



$$1 + 6 + 3 = 10$$

小課題1

- $M \leq 1,000$
辺が少ない。
- すべての辺について、その辺を反転したグラフを作り、それぞれの
場合で $1 \rightarrow N$ の最短路長と
 $N \rightarrow 1$ の最短路長を求める。

小課題1

最短路長の求め方

ダイクストラ法

(蟻本やネットなどで調べてください)

小課題1

ダイクストラ法の計算量:

$O(N^2 + M)$ (毎ステップ、すべての頂点を見る)

$O(M \log M)$ (priority_queue とかを使う)

小課題1 ($N \leq 200, M \leq 1,000$)

疎なグラフなので、 $O(M \log M)$ の方が速い

小課題2~ ($N \leq 200, M \leq 50,000$)

密なグラフなので、 $O(N^2 + M)$ の方が速い

小課題1

- $M \leq 1,000$
辺が少ない。
- すべての辺について、その辺を反転したグラフを作り、それぞれのグラフについて
 $1 \rightarrow N$ の最短路長と $N \rightarrow 1$ の最短路長を求める。
- 毎回ダイクストラ法を使うと、
 $O(M^2 \log M)$ または $O(M(N^2 + M))$

5点

小課題3

- 移動コスト0
- どの辺を反転させれば、1とNとが互いに行き来できるようになるか。
- 答えは反転コスト D の最小値

小課題3

解法案: 反転コストが小さいものから順に反転。
1とNとが互いに行き来できれば、
その反転コストを出力。

愚直にやって、1辺あたり($O(M)$) $\rightarrow$ TLE

小課題3

毎回 $O(M)$ かけて探索したくない。

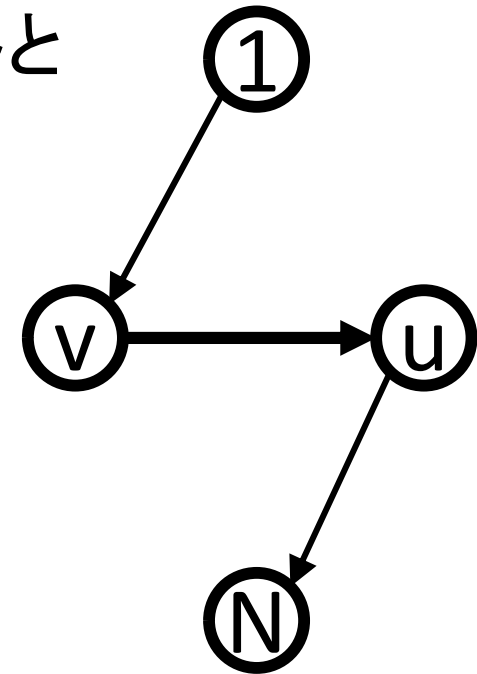
もともと $1 \rightarrow N$ の経路が存在しないときに

辺 $u \rightarrow v$ を反転させて $1 \rightarrow N$ へ行けるようになる条件:

「反転させる前のグラフで、 $1 \rightarrow v$ の経路と
 $u \rightarrow N$ の経路が存在する」

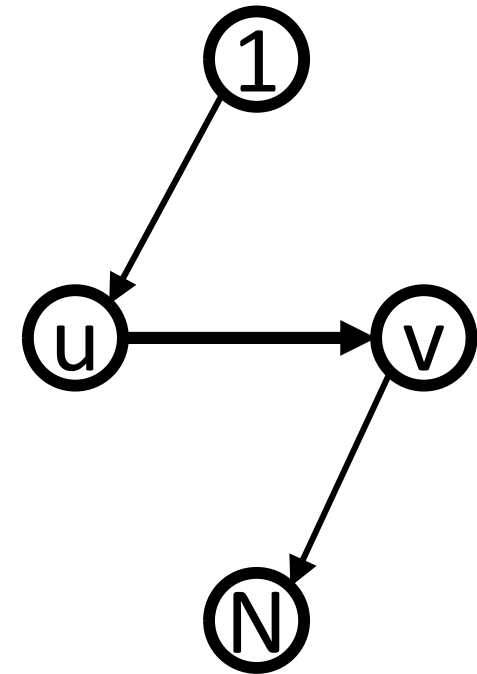
これは、前もって

- $1 \rightarrow v$ の経路が存在するか、
 - $u \rightarrow N$ の経路が存在するか、
- を計算しておけば、 $O(1)$ で判定できる。



小課題3

※もともと $1 \rightarrow N$ の経路が存在するときには
この条件は使えないことに注意



小課題3

解法案

- 1から到達できる頂点集合
 - Nまで到達できる頂点集合
 - Nから到達できる頂点集合
 - 1まで到達できる頂点集合
- を予め計算する。

小課題3

解法案

A. 1とNとが互いに行き来できる場合

答えは0

B. 1とNとの間にどちら向きにも経路が無い場合

先ほどの条件が使えて、辺ごとに $O(1)$ で判定できる。

C. $N \rightarrow 1$ の向きにだけ経路がある場合

反転コストの小さい順に辺を見ていく。

$1 \rightarrow N$ の向きには先ほどの条件が使える。

$1 \rightarrow N$ に経路が存在する場合のみ、 $N \rightarrow 1$ の経路があるか判定する。→TLE??

小課題3

C. $N \rightarrow 1$ の向きにだけ経路がある場合

反転コストの小さい順に辺を見ていく。

$1 \rightarrow N$ の向きには先ほどの条件が使える。

$1 \rightarrow N$ に経路が存在する場合のみ、 $N \rightarrow 1$ の経路があるか判定する。

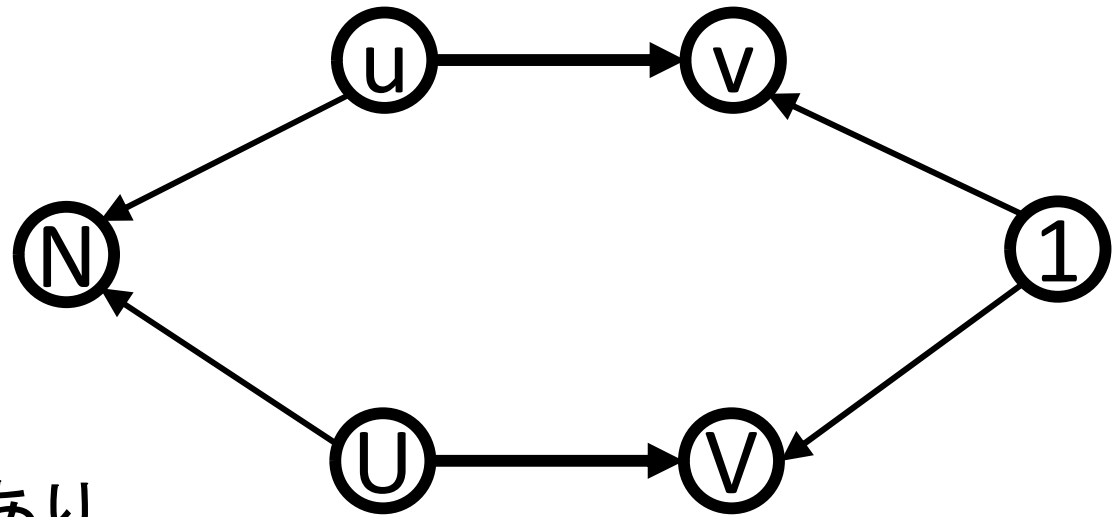
実は、この判定を2回やれば
少なくとも1回は経路がある。

21点

小課題3

証明

$N \rightarrow 1$ への経路があり
 $1 \rightarrow N$ への経路が存在せず
2つの辺 $u \rightarrow v$ と $U \rightarrow V$ について、それぞれ反転させたときに $1 \rightarrow N$ への経路が新たに生まれ、逆に $N \rightarrow 1$ の経路が無くなるものとする。



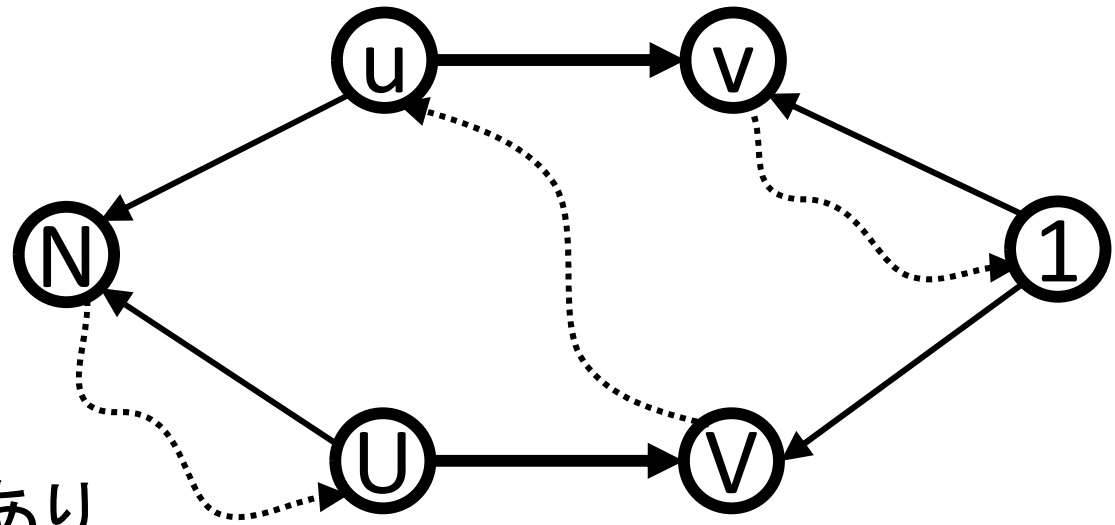
小課題3

証明

$N \rightarrow 1$ への経路があり
 $1 \rightarrow N$ への経路が存在せず
2つの辺 $u \rightarrow v$ と $U \rightarrow V$ について、それぞれ反転させたときに $1 \rightarrow N$ への経路が新たに生まれ、逆に $N \rightarrow 1$ の経路が無くなるものとする。

反転させたことにより $N \rightarrow 1$ の経路が無くなるので、 $N \rightarrow 1$ の経路は $U \rightarrow V$ と $u \rightarrow v$ の両方の辺を通る。

このとき上図のようになって $1 \rightarrow N$ の経路があることになり矛盾。

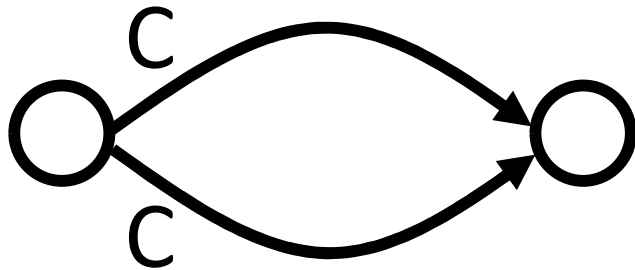


小課題2

- 辺数 M は偶数
- $U_{2i-1} = U_{2i}, V_{2i-1} = V_{2i}, C_{2i-1} = C_{2i}$
→ 辺が2本ずつ重複している
※ D は一緒とは限らないので注意

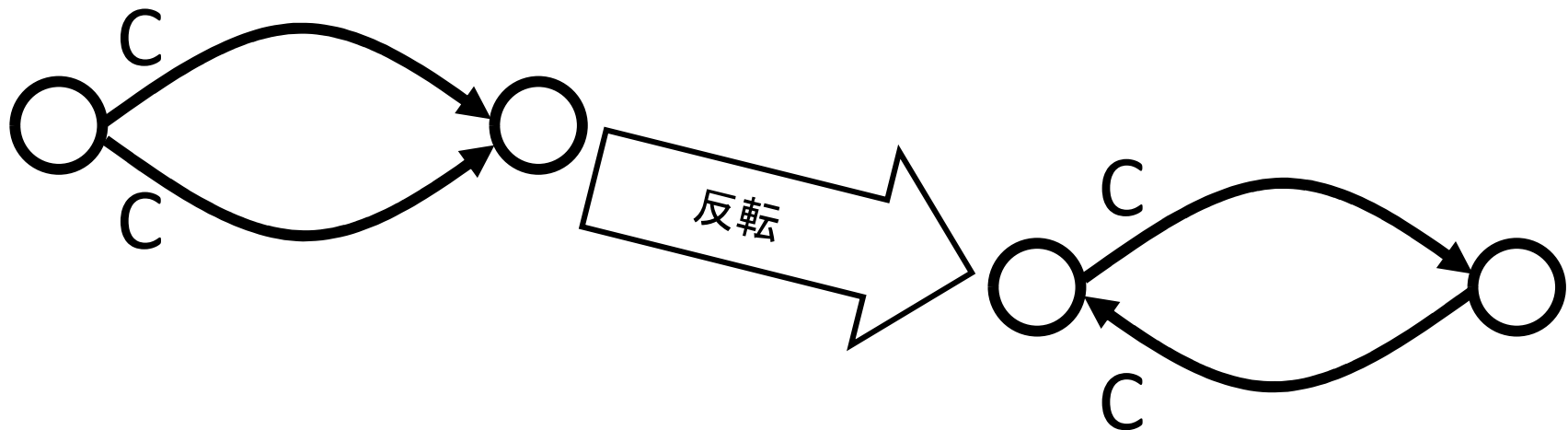
小課題2

- 最短路を考える上では...
辺を反転させる = 逆向きの辺を増やす



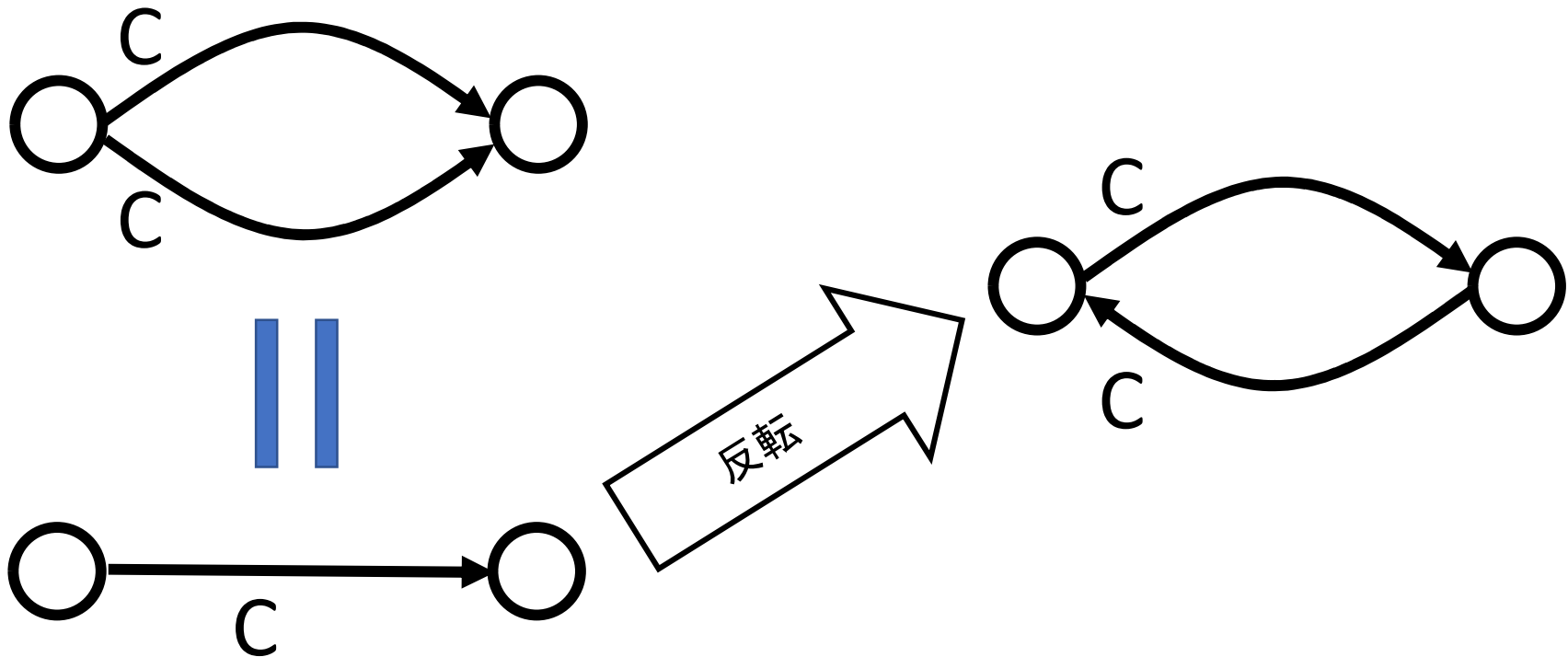
小課題2

- 最短路を考える上では...
辺を反転させる = 逆向きの辺を増やす



小課題2

- 最短路を考える上では...
辺を反転させる = 逆向きの辺を増やす



小課題2

- 最短路を考える上では...
辺を反転させる = 逆向きの辺を増やす
- うまくグラフを前処理することで、
辺を1本増やしたときの最短路長を
高速に計算したい。

小課題2

- 辺を1本増やしたときの最短路長
= ①増やした辺を通らないときの最短路長
②増やした辺を必ず通るときの最短路長
のうち小さい方
- ①はすべての辺について共通。
前計算すればよい。

小課題2

②増やした辺を必ず通るときの最短路長



小課題2

②増やした辺を必ず通るときの最短路長

このとき通る $1 \rightarrow v$ の経路について

- $1 \rightarrow v$ の最短路になっている
- 増やした辺($v \rightarrow u$ の辺)を含まない



小課題2

②増やした辺を必ず通るときの最短路長

このとき通る $1 \rightarrow v$ の経路について

- $1 \rightarrow v$ の最短路になっている
- 増やした辺($v \rightarrow u$ の辺)を含まない

予め $1 \rightarrow v$ の最短路長を計算すれば良い



小課題2

各辺を反転させたときの 1→Nの最短路を高速に求める方法

辺を反転させずに

- 1から各頂点までの最短路
- 各頂点からNまでの最短路

の2つを予め求める。これを $\text{dist0}[x][y]$ などと書く。

それぞれ順方向と逆方向の辺を使い、
ダイクストラ法1回ずつ。 $O(N^2 + M)$

小課題2

各辺を反転させたときの
1→Nの最短路を高速に求める方法

辺 $u \rightarrow v$ (C) を反転させたときの1→Nの最短路長は

① $\text{dist0}[1][N]$

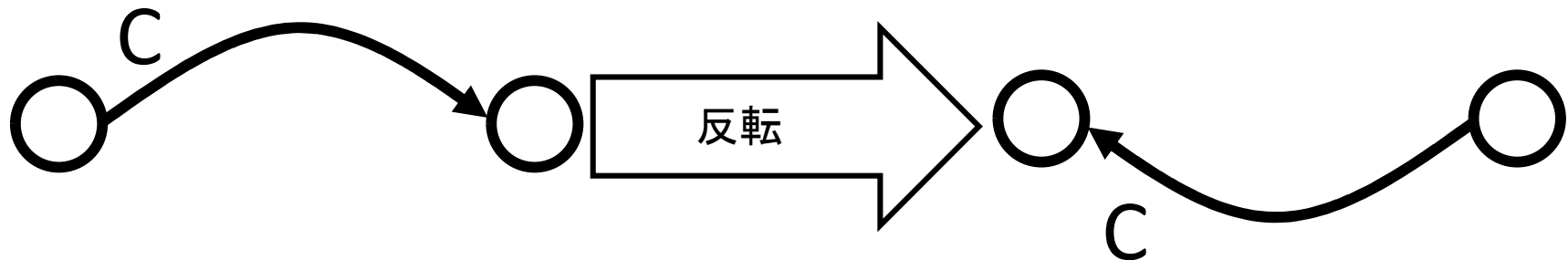
② $\text{dist0}[1][v] + \text{dist0}[u][N] + C$

のうち小さい方。

各辺について $O(1)$ **11点**

小課題4(完答)

- 小課題2の解法の問題点
辺を反転させたときに消滅する辺があり、
これに対応できない。



小課題4(完答)

- 以降、反転させる辺を固定して考える。
- 小課題2の解法の問題点
 辺を反転させたときに消滅する辺があり、
 これに対応できない。
- 逆に言えば、消滅する辺の影響を受けないような
 場合は、小課題2の解法がそのまま適用できる。

小課題2(再掲)

各辺を反転させたときの
1→Nの最短路を高速に求める方法

辺 $u \rightarrow v$ (C) を反転させたときの1→Nの最短路長は

① $\text{dist0}[1][N]$

② $\text{dist0}[1][v] + \text{dist0}[u][N] + C$

のうち小さい方。

各辺について $O(1)$ **11点**

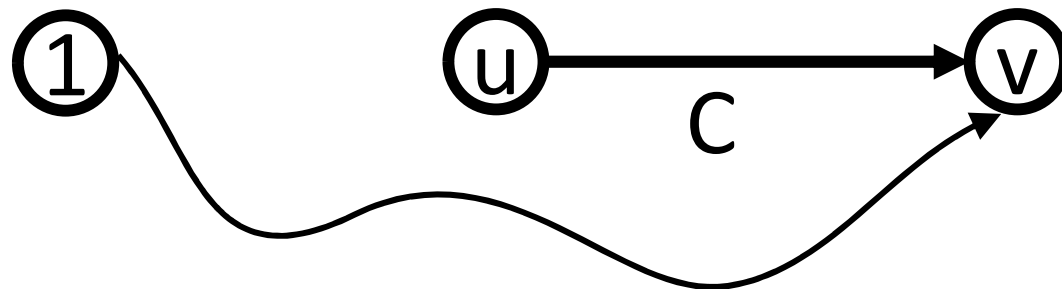
小課題4(完答)

- 最短路長
= ①増やした辺を通らないときの最短路長
②増やした辺を必ず通るときの最短路長
のうち小さい方
 - $1 \rightarrow N$ の最短路に消滅する辺を含まないとき
① = $\text{dist0}[1][N]$
 - $1 \rightarrow v$ や $u \rightarrow N$ の最短路に(以下略)
② = $\text{dist0}[1][v] + \text{dist0}[u][N] + C$
- どちらも満たすときは小課題2と同様に $O(1)$

小課題4(完答)

$\text{dist0}[1][N]$, $\text{dist0}[1][v]$, $\text{dist0}[u][N]$ の経路に
消滅する辺を含むときが問題

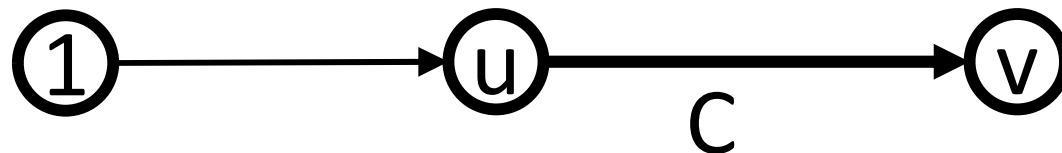
考察 ($1 \rightarrow v$ の最短路に消滅する辺を含むとき)
このとき $1 \rightarrow v$ の最短路中で u を通っている。



小課題4(完答)

$\text{dist0}[1][N]$, $\text{dist0}[1][v]$, $\text{dist0}[u][N]$ の経路に
消滅する辺を含むときが問題

考察 ($1 \rightarrow v$ の最短路に消滅する辺を含むとき)
このとき $1 \rightarrow v$ の最短路中で u を通っている。

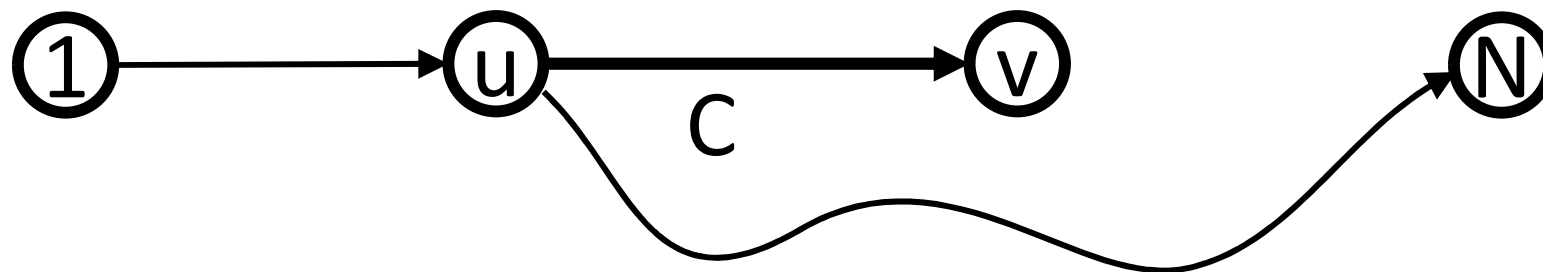


小課題4(完答)

$\text{dist0}[1][N]$, $\text{dist0}[1][v]$, $\text{dist0}[u][N]$ の経路に
消滅する辺を含むときが問題

考察 ($1 \rightarrow v$ の最短路に消滅する辺を含むとき)

$$\begin{aligned} \text{dist0}[1][v] + \text{dist0}[u][N] &\geq \text{dist0}[1][u] + \text{dist0}[u][N] \\ &\geq \text{dist0}[1][N] \end{aligned}$$



小課題4(完答)

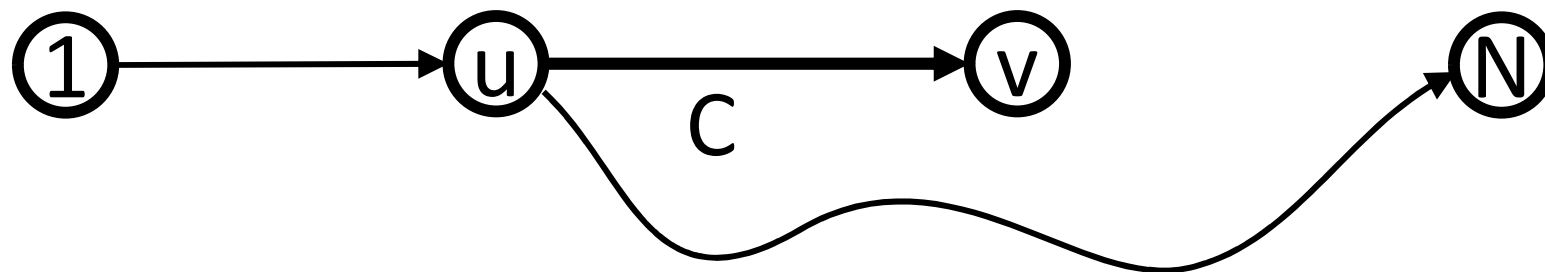
考察 ($1 \rightarrow v$ の最短路に消滅する辺を含むとき)

$$\text{dist0}[1][v] + \text{dist0}[u][N] \geq \text{dist0}[1][u] + \text{dist0}[u][N] \geq \text{dist0}[1][N]$$

1辺削除したときの距離を $\text{dist1}[x][y]$ と書くと

$$\textcircled{2} = \text{dist1}[1][v] + \text{dist1}[u][N] + C$$

$$\geq \text{dist0}[1][v] + \text{dist0}[u][N] + C \geq \text{dist0}[1][N]$$



小課題4(完答)

考察 ($\text{dist0}[1][v]$ の経路に消滅する辺を含むとき)

$$\text{dist0}[1][v] + \text{dist0}[u][N] \geq \text{dist0}[1][u] + \text{dist0}[u][N] \geq \text{dist0}[1][N]$$

1辺削除したときの距離を $\text{dist1}[x][y]$ と書くと

$$\textcircled{2} = \text{dist1}[1][v] + \text{dist1}[u][N] + C$$

$$\geq \text{dist0}[1][v] + \text{dist0}[u][N] + C \geq \text{dist0}[1][N]$$

$\text{dist0}[1][N]$ の経路に消滅する辺を含まないときは

$$(\text{右辺}) = \textcircled{1}$$

となって小課題2のアルゴリズムでOK

小課題4(完答)

分かったこと

注目する辺が $1 \rightarrow v$ や $u \rightarrow N$ の最短路に含まれていても、 $1 \rightarrow N$ の最短路に含まれていなければ
小課題2の解法でOK

小課題4(完答)

分かったこと

注目する辺が $1 \rightarrow v$ や $u \rightarrow N$ の最短路に含まれていても、 $1 \rightarrow N$ の最短路に含まれていなければ
小課題2の解法でOK

つまり...

- 注目する辺が $1 \rightarrow N$ の最短路に含まれないとき
小課題2の解法でOK
- 注目する辺が $1 \rightarrow N$ の最短路に含まれるとき
これが問題

小課題4(完答)

注目する辺が $1 \rightarrow N$ の最短路に含まれるとき
($1 \rightarrow N$ の最短路上の辺を反転させるとき)

- $1 \rightarrow N$ の最短路には高々 N 本しか辺が含まれないので、これは高々 N 回しか起こらない。
- この場合は毎回ダイクストラ法をやっても、合計 $O(N(N^2 + M))$ となり、間に合う。

小課題4(完答)

各辺を反転させたときの
1→Nの最短路を高速に求める方法

1→Nの最短路を1つ求め、それに含まれる辺を
予め記憶する。

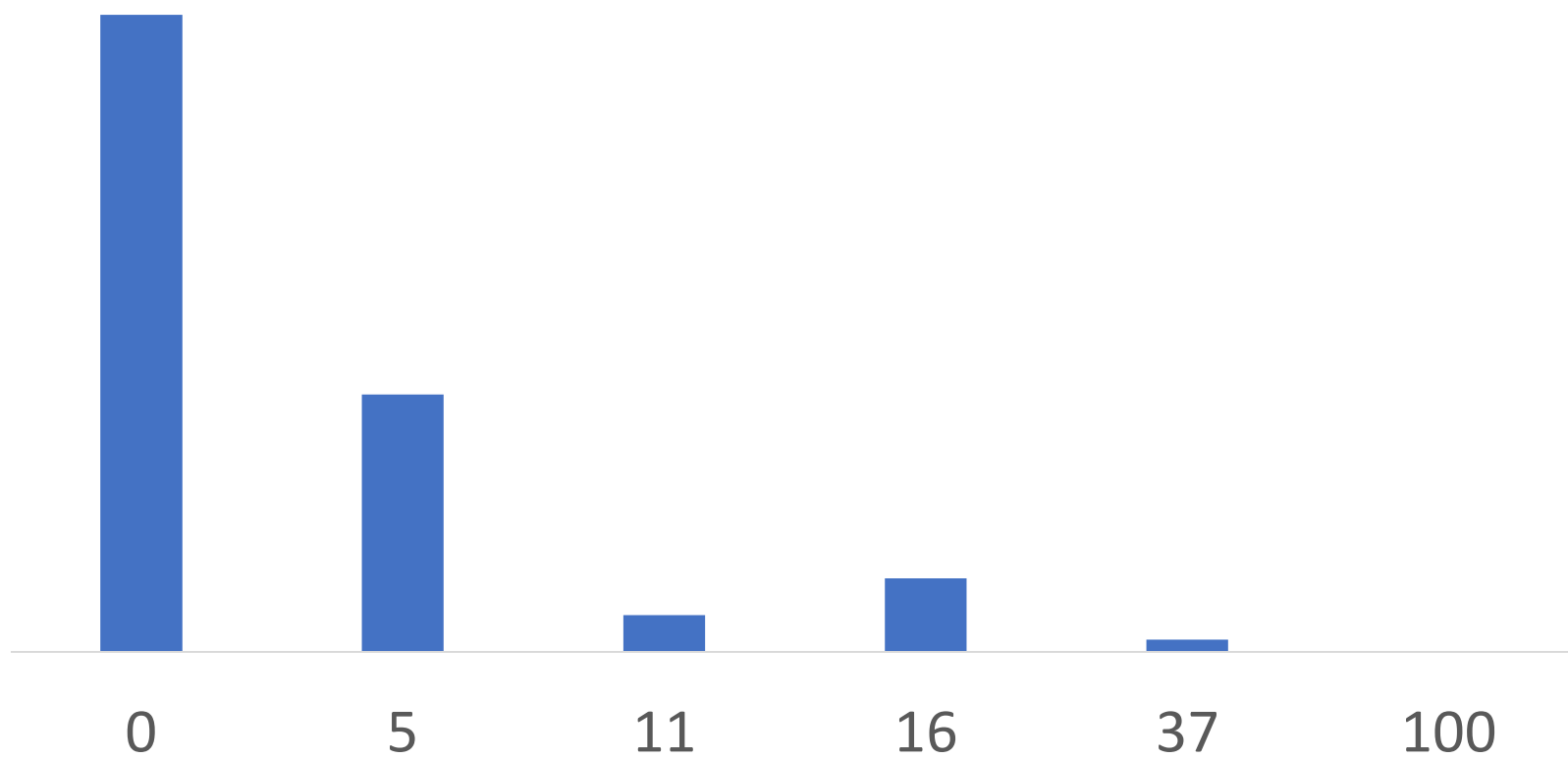
- 最短路上の辺については、毎回ダイクストラ
- それ以外の辺については、小課題2と同様

$O(N(N^2 + M))$ **完答**

小課題4(完答)

※ $1 \rightarrow N$ の経路が少なくとも1つ存在する前提で解説をしましたが、経路が存在しない場合も全く同様の方法で解けます。

得点分布



JOI2019/2020 本選第五問

火事(Fire) 解説

解説者：小倉拳

問題概要

- 要素数 N の数列 A が与えられる
- 各要素の値は時間が1過ぎると左の要素とのmaxに変わる
- 以下の形式のクエリに Q 個答えよ
- 時刻 T における $A_L + \dots + A_R$ の総和を求めよ

制約

- $N, Q \leq 200,000$
- $A_i \leq 1,000,000,000$

例

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3						
時刻4						
時刻5						

例

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1	8	8	7	7	4	9
時刻2						
時刻3						
時刻4						
時刻5						

例

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1	8	8	7	7	4	9
時刻2	8	8	8	7	7	9
時刻3	8	8	8	8	7	9
時刻4	8	8	8	8	8	9
時刻5	8	8	8	8	8	9

例

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1	8	8	7	7	4	9
時刻2	8	8	8	7	7	9
時刻3	8	8	8	8	7	9
時刻4	8	8	8	8	8	9
時刻5	8	8	8	8	8	9

- クエリ：時刻3の区画3から区画6の総和は？
- $8+8+7+9 = 32$

小課題

- 小課題1. $N, Q \leq 200$
- 小課題2. クエリの T がすべて同じ
- 小課題3. 各クエリについて $R = L$
- 小課題4. A の要素の値は1か2
- 小課題5. 追加制約なし

まず思うこと

- オフラインクエリ問題なのでクエリは好きな順番に答えてよい
 - T が小さい順？ L が小さい順？ R が小さい順？

小課題1. $N, Q \leq 200$

小課題1. $N, Q \leq 200$

- 愚直にシミュレーションしてよい

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1	8	8	7	7	4	9
時刻2	8	8	8	7	7	9
時刻3	8	8	8	8	7	9
時刻4	8	8	8	8	8	9
時刻5	8	8	8	8	8	9

小課題1. $N, Q \leq 200$

- 愚直にシミュレーションしてよい
- 時刻 t の区画 i の火の強さをすべてあらかじめ計算して、クエリに愚直に答える
 - $O(N^2 + QN)$
 - 累積和などを使えば $O(N^2 + Q)$
- 1点獲得 🙌

小課題1. $N, Q \leq 200$ (別解)

- ある時刻のある区画をピンポイントで求めたい

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1	8	8	7	7	4	9
時刻2	8	8	8	7	7	9
時刻3	8	8	8	8	7	9
時刻4	8	8	8	8	8	9
時刻5	8	8	8	8	8	9

小課題1. $N, Q \leq 200$ (別解)

- 関係あるところ

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1			7	7	4	
時刻2				7	7	
時刻3					7	
時刻4						
時刻5						

小課題1. $N, Q \leq 200$ (別解)

- 時刻0のある区間のmaxと一致する

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3					7	
時刻4						
時刻5						

- 時刻 t の区画 i の火の強さは、時刻0の区画 $i - t$ から区画 i までの火の強さの最大値

小課題1. $N, Q \leq 200$ (別解)

- 時刻 t の区画 i の火の強さは、時刻 0 の区画 $i - t$ から区画 i までの火の強さの最大値,
- 毎回クエリ内のすべての区間の火の強さを求めても良い
 - $O(N^2Q)$
- 1点獲得 🙌

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3						
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3	8					
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3	8	8				
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3	8	8	8			
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3	8	8	8	8		
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3	8	8	8	8	7	
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- ある一列だけしかクエリが来ない
- そこだけ表を作ればOK!

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3	8	8	8	8	7	9
時刻4						
時刻5						

小課題2. クエリの T がすべて同じ

- いくつかの区間についてその中の最大値が求まればよい
 - **RMQ? SparseTable?** まあそれでもOK
- 区間が単調に右にずれていく
 - **Slide最大値** が使える
 - $O(N \log N)$ や $O(N)$ で時刻 T の各区画の火の強さがわかる
 - 累積和を取ることで $O(Q)$ ですべてのクエリに応答可能
- 6点獲得 🙌🙌

小課題3. 各クエリについて $R = L$

- クエリは以下のような形式だけ

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	8	6	7	4	2	9
時刻1						
時刻2						
時刻3					7	
時刻4						
時刻5						

小課題3. 各クエリについて $R = L$

- 時刻0のいろんな区間の最大値がわかればよい
 - ズバリ**RMQ**(Range Maximum Query)
 - **SegmentTree**や**SparseTable**が使える
 - $O(\log N)$ や $O(1)$ で好きな時刻の好きな一区画の火の強さがわかる
 - $O((N + Q \log N))$ や $O((Q + N \log N))$
- 7点獲得 🙌🙌🙌

小課題4. A の要素の値は1か2

- まずは絵を描く

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	2	1	1	1	2	1
時刻1	2	2	1	1	2	2
時刻2	2	2	2	1	2	2
時刻3	2	2	2	2	2	2
時刻4	2	2	2	2	2	2
時刻5	2	2	2	2	2	2

小課題4. A の要素の値は1か2

- 各区画の火の強さはあるタイミングまで1で、それ以降ずっと2

	区画1	区画2	区画3	区画4	区画5	区画6
時刻0	2	1	1	1	2	1
時刻1	2	2	1	1	2	2
時刻2	2	2	2	1	2	2
時刻3	2	2	2	2	2	2
時刻4	2	2	2	2	2	2
時刻5	2	2	2	2	2	2

小課題4. A の要素の値は1か2

- 各区画の火の強さはあるタイミングまで1で、それ以降ずっと2
 - 時刻順にクエリを捌けば、区画の火の強さの変化は
全体で N 回以下
 - 一点更新、区間和取得を速く処理したい...
 - → **SegmentTree, BIT**
 - $O((N + Q) \log N)$
- 6点獲得 🙌🙌🙌🙌

小課題5. 追加制約なし (🔥本番🔥)

小課題5. 追加制約なし (🔥本番🔥)

やること

- クエリ先読み
- 時刻0の各区画の火の強さがどこまで波及するか考える **New !**
- 平行四辺形を線に分解する **What?!**
- データ構造で処理

各区画がどこまで波及するか考える

- 各時刻の各区画の火の強さを並べた絵を書く

3	1	4	1	5	9	2	6	5	3	5	8	9
3	3	4	4	5	9	9	6	6	5	5	8	9
3	3	4	4	5	9	9	9	6	6	5	8	9
3	3	4	4	5	9	9	9	9	6	6	8	9
3	3	4	4	5	9	9	9	9	9	6	8	9
3	3	4	4	5	9	9	9	9	9	9	8	9
3	3	4	4	5	9	9	9	9	9	9	9	9

各区画がどこまで波及するか考える

- 時刻0のどの区画由来かで色を塗る

3	1	4	1	5	9	2	6	5	3	5	8	9
3	3	4	4	5	9	9	6	6	5	5	8	9
3	3	4	4	5	9	9	9	6	6	5	8	9
3	3	4	4	5	9	9	9	9	6	6	8	9
3	3	4	4	5	9	9	9	9	9	6	8	9
3	3	4	4	5	9	9	9	9	9	9	8	9
3	3	4	4	5	9	9	9	9	9	9	9	9

各区画がどこまで波及するか考える

- 時刻0のどの区画由来かで色を塗る

3	1	4	1	5	9	2	6	5	3	5	8	9
3	3	4	4	5	9	9	6	6	5	5	8	9
3	3	4	4	5	9	9	9	6	6	5	8	9
3	3	4	4	5	9	9	9	9	6	6	8	9
3	3	4	4	5	9	9	9	9	9	6	8	9
3	3	4	4	5	9	9	9	9	9	9	8	9
3	3	4	4	5	9	9	9	9	9	9	9	9

- 平行四辺形っぽい！

平行四辺形になります

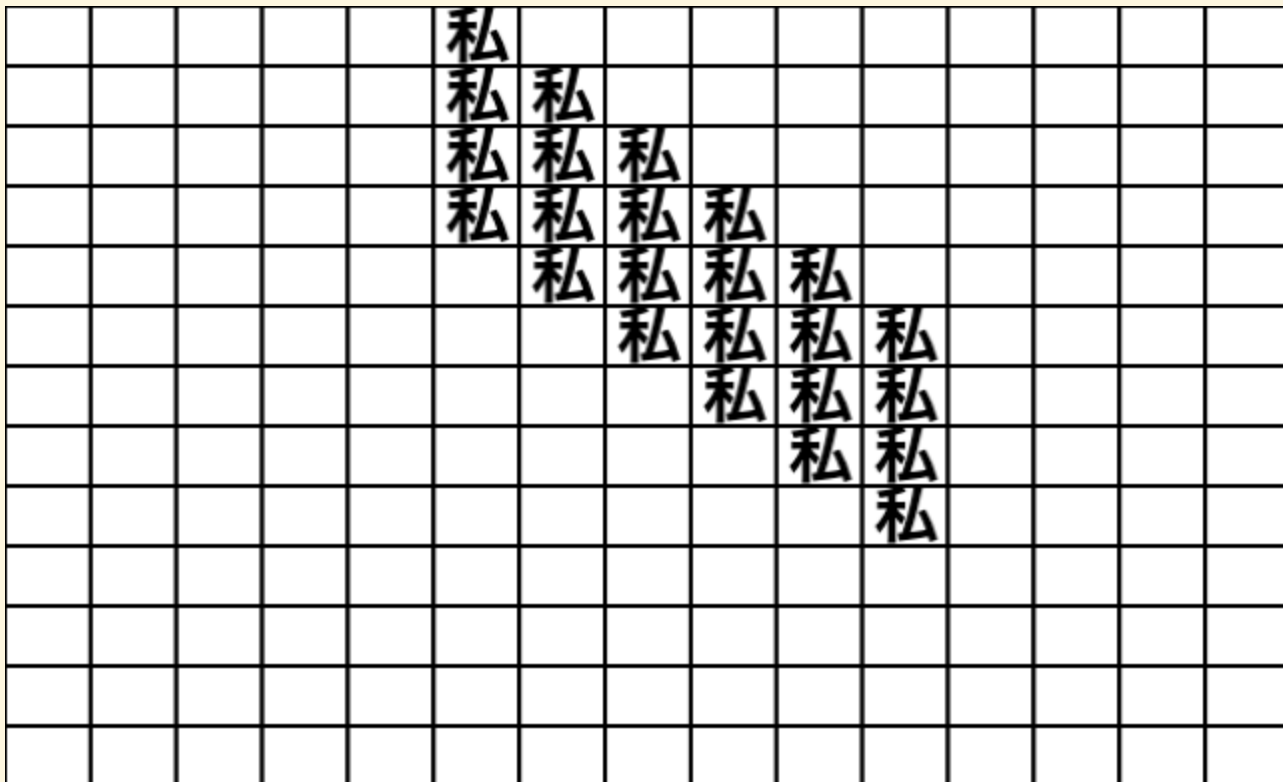
- 区画 i より**左**で区画 i より大きくなる初めの区画 L とする
- 区画 i より**右**で区画 i より大きくなる初めの区画 R とする
- 以下の区画が時刻 0 の区画 i と同じ値になる

	L	小	小	小	私	小	小	小	小	小	R			
		L	小	小	私	私	小	小	小	小	R			
			L	小	私	私	私	小	小	小	R			
				L	私	私	私	私	小	小	R			
					L	私	私	私	私	小	R			
						L	私	私	私	私	R			
							L	私	私	私	R			
								L	私	私	R			
									L	私	R			
										L	R			

平行四辺形いやだな

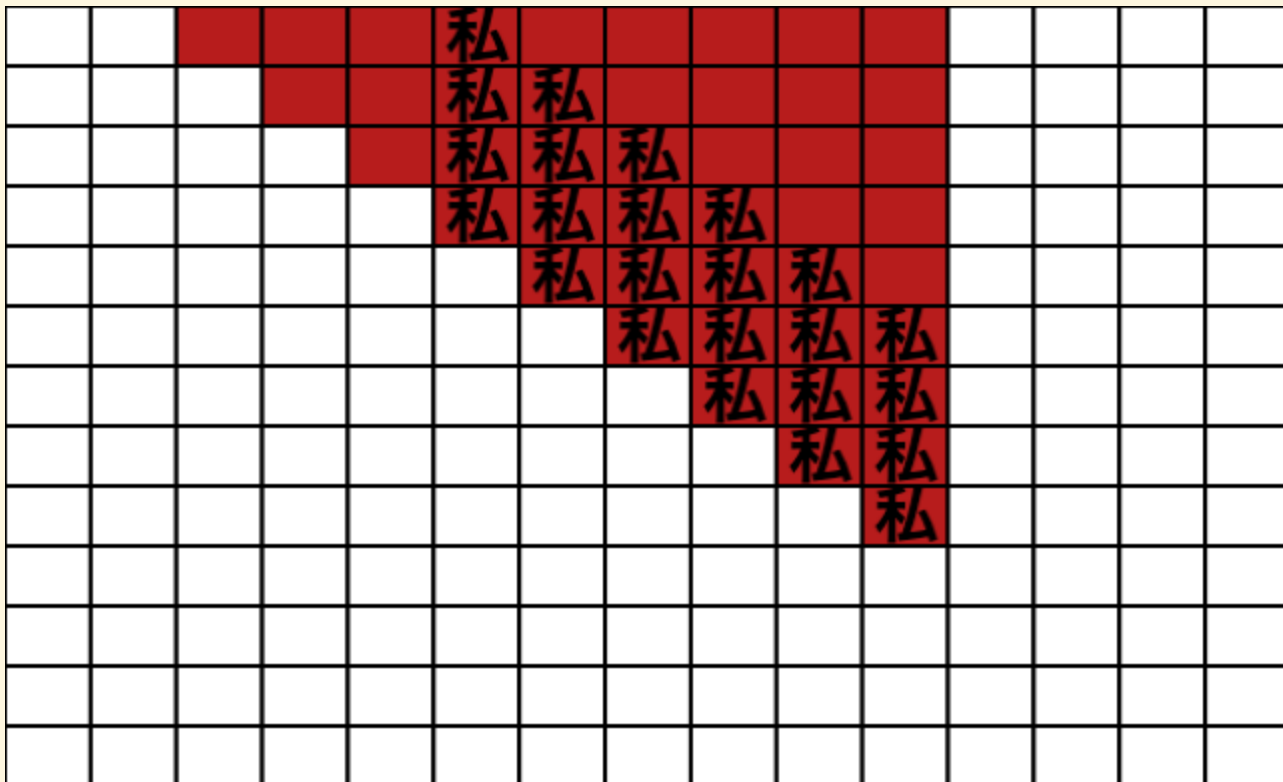
- 平行四辺形状に値を足せばOKだが、そんなデータ構造知らない
 - 分解！
- 平行四辺形への足しこみは三角形の足しこみに分解できる！

平行四辺形を三角形であらわす



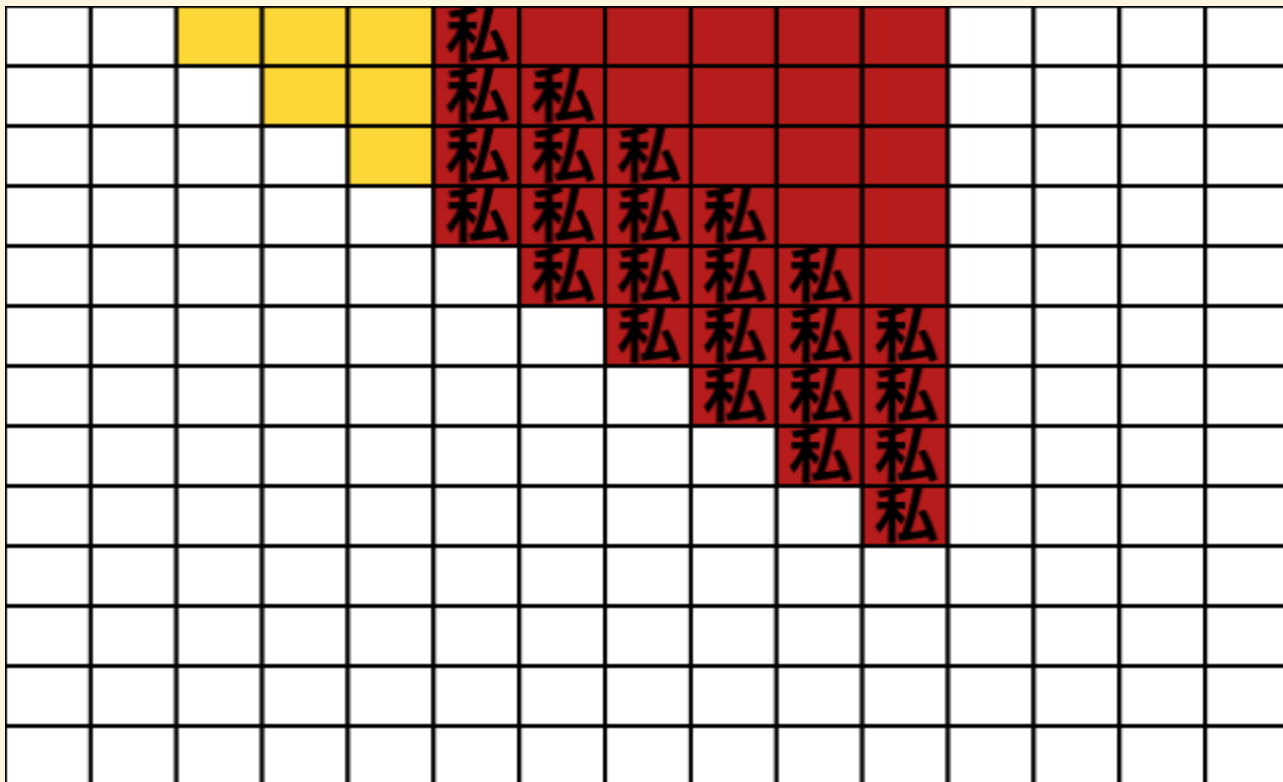
- 私にだけxを足したいです

平行四辺形を三角形であらわす



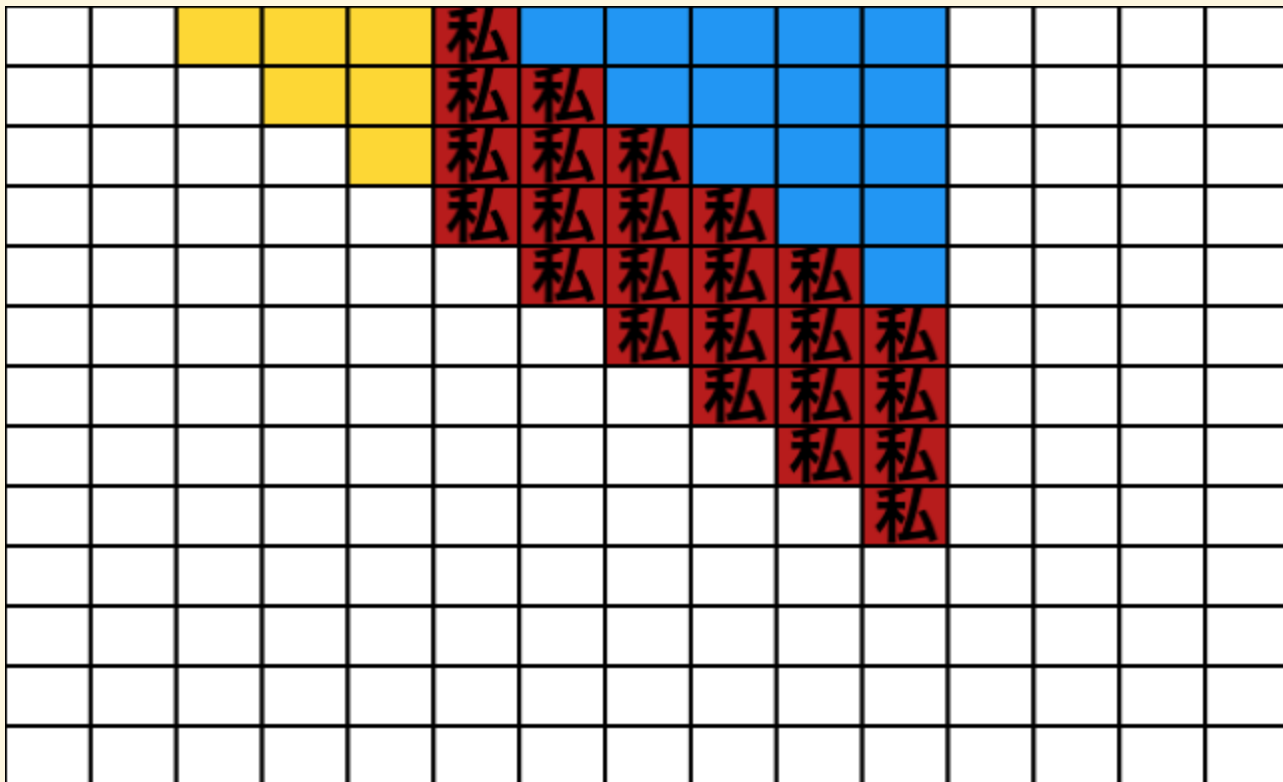
- 赤にxを足します

平行四辺形を三角形であらわす



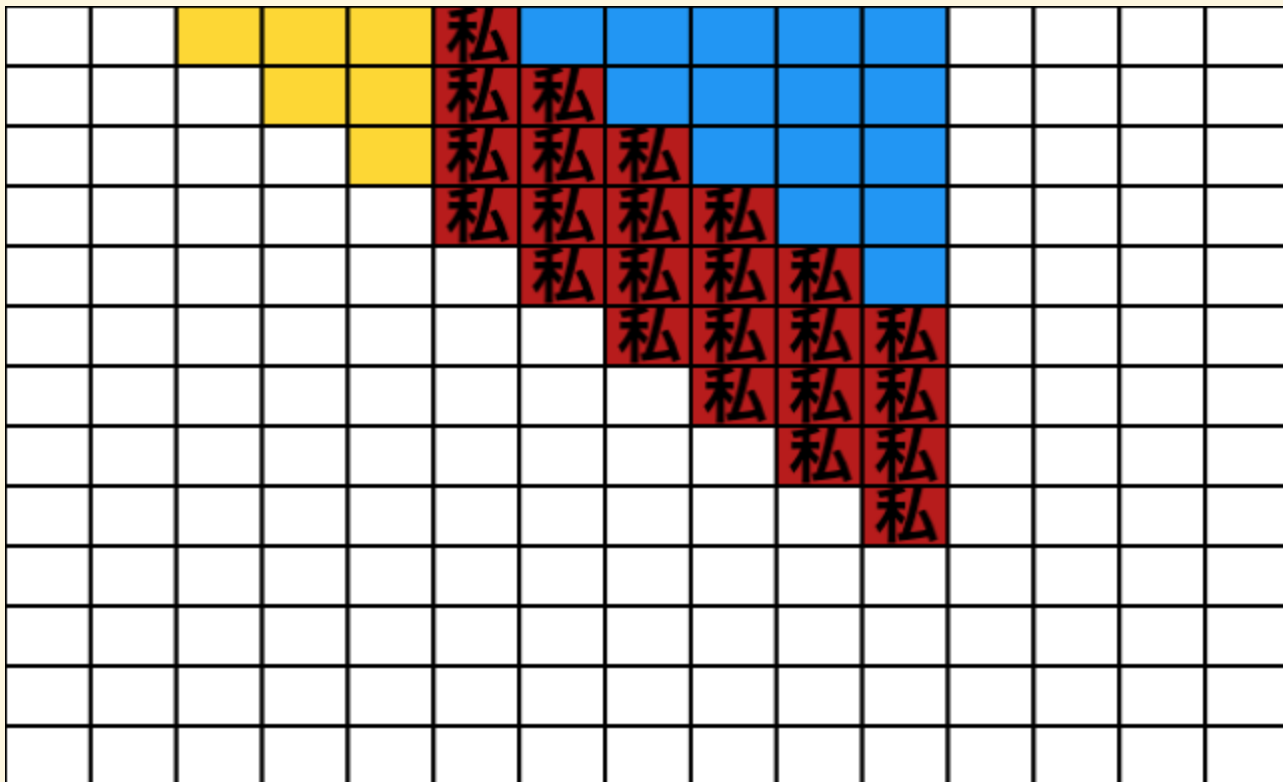
- 黄色に-xを足します

平行四辺形を三角形であらわす



- 青に-xを足します

平行四辺形を三角形であらわす



- 完成👏

三角形いやだな

三角形は以下の二つに分解できます

- ある斜め線より右側に一様に足す
- ある縦線より右側に一様に足す

三角形を分解

		私	私	私	私	私	私	私	私	私				
			私	私	私	私	私	私	私	私				
				私	私	私	私	私	私	私				
					私	私	私	私	私	私				
						私	私	私	私	私				
							私	私	私	私				
								私	私	私				
									私	私				
										私				
											私			

- 私にだけxを足したいです

三角形を分解

		私	私	私	私	私	私	私	私	私				
			私	私	私	私	私	私	私	私				
				私	私	私	私	私	私	私				
					私	私	私	私	私	私				
						私	私	私	私	私				
							私	私	私	私				
								私	私	私				
									私	私				
										私				
											私			

- 赤にxを足します.

三角形を分解

		私	私	私	私	私	私	私	私	私				
			私	私	私	私	私	私	私	私				
				私	私	私	私	私	私	私				
					私	私	私	私	私	私				
						私	私	私	私	私				
							私	私	私	私				
								私	私	私				
									私	私				
										私				
											私			

- 青に-xを足します

三角形を分解

		私	私	私	私	私	私	私	私	私				
			私	私	私	私	私	私	私	私				
				私	私	私	私	私	私	私				
					私	私	私	私	私	私				
						私	私	私	私	私				
							私	私	私	私				
								私	私	私				
									私	私				
										私				

- 完成 🙌

こんなデータ構造が欲しい

- ある縦線より右側に一様に足す
- ある斜め線より右側に一様に足す
- 区間の総和取る

こんなデータ構造が欲しい

- ある縦線より右側に一様に足す
- ある斜め線より右側に一様に足す
- 区間の総和取る

そんなデータ構造ある？

こんなデータ構造が欲しい

- ある縦線より右側に一様に足す
- ある斜め線より右側に一様に足す
- 区間の総和取る

そんなデータ構造ある？

- 私は思いつきませんでした. . .

2つのデータ構造に分ける

縦線用のデータ構造

- ある縦線より右側に一様に足す
- 区間の総和取る

斜め線用のデータ構造

- ある斜め線より右側に一様に足す
- 区間の総和取る

に分解して，クエリの和を取ればOK！

縦線用データ構造

- 小課題4と同様に時系列順にクエリを処理することを考える
- あるタイミングで一様に x を足したり引いたりするだけ
- 区間の総和も取りたい

SegmentTreeを1つ or **BIT**を3つ

縦線用データ構造

- 小課題4と同様に時系列順にクエリを処理することを考える
- あるタイミングで一様に x を足したり引いたりするだけ
- 区間の総和も取りたい

SegmentTreeを1つ or **BIT**を3つ

- 時間の関係でデータ構造の詳細は割愛 

縦線用データ構造

- 小課題4と同様に時系列順にクエリを処理することを考える
- あるタイミングで一様にxを足したり引いたりするだけ
- 区間の総和も取りたい

SegmentTreeを1つ or BITを3つ

The screenshot shows a YouTube video player interface. The video title is "木マスター養成講座 4.1. Segment木ってなに？導入編". The video player shows a thumbnail with the text "かつっぱの木マスター養成講座" and "4. Segment木ってなに？". The video player controls show the video is at 0:00 / 20:00. The video description includes "【木マスター養成講座】 4-1. Segment木ってなに？導入編 【競プロかつぱ】" and "222 回視聴・2020/01/15". The right sidebar shows a list of video thumbnails and titles, including "【木マスター養成講座】 4-1. Segment木ってなに？導入編", "【木マスター養成講座】 4-2. Segment木ってなに？なん...", "【木マスター養成講座】 7-1. Splay木ってなに？説明編...", "【木マスター養成講座】 7-2. Splay木ってなに？実装編 1...", "【木マスター養成講座】 7-3. Splay木ってなに？実装編 2...", and "【木マスター養成講座】 8-1. Link-Cut木ってなに？解説編...".

"JOI2019/2020 本選5 火事(fire)"

斜め線用データ構造

- 縦線用データ構造を用意する
- 足すときは縦に足す
- クエリする範囲を適切にずらせばOK

				Q1	Q1	Q1	Q1	Q1						
										Q2	Q2	Q2	Q2	
			Q3	Q3	Q3									
						Q4	Q4	Q4	Q4	Q4	Q4			
								Q5	Q5	Q5	Q5			

斜め線より右側に一樣に足す

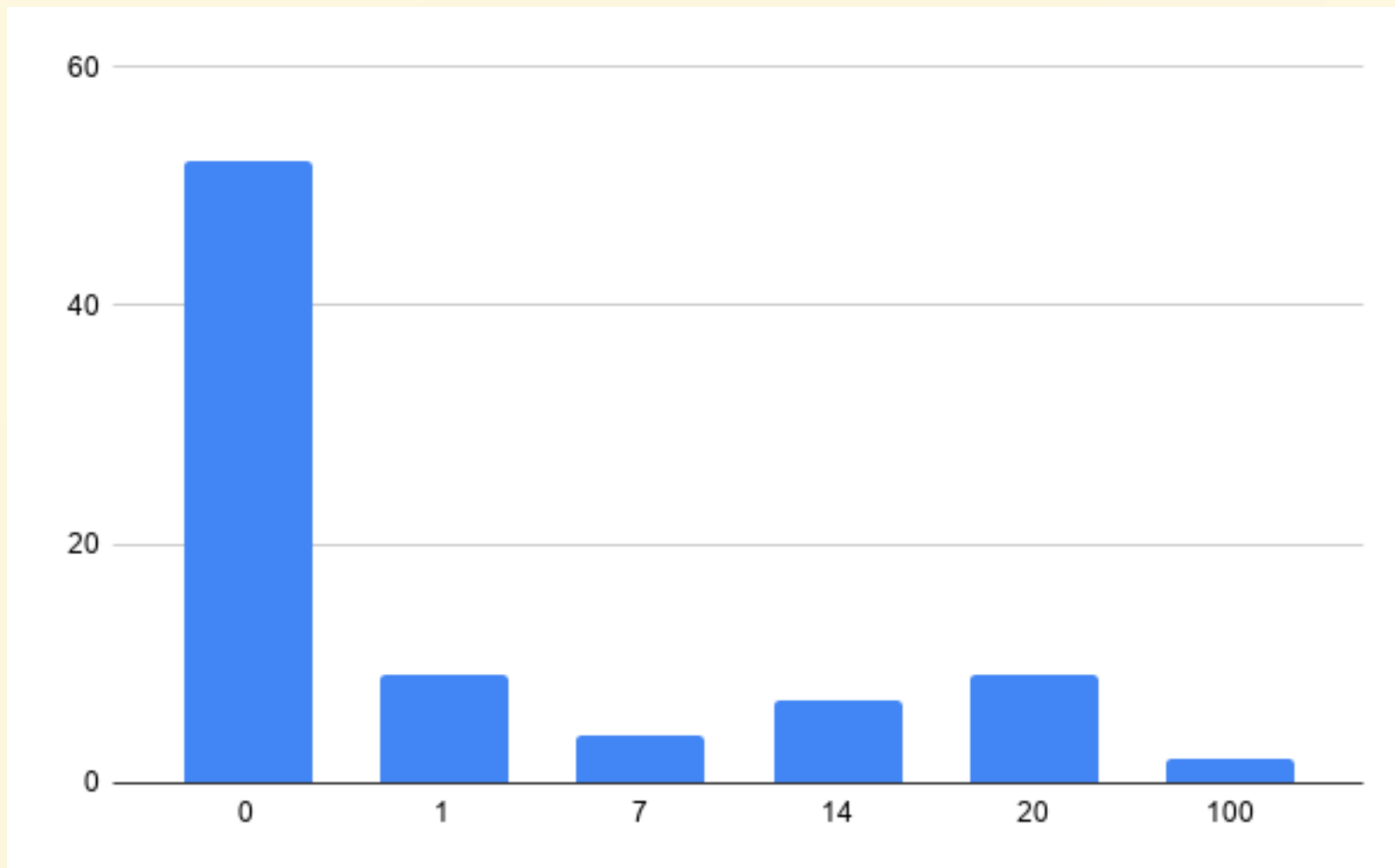
- 縦線用データ構造を用意する
- 足すときは縦に足す
- クエリする範囲を適切にずらせばOK

			Q1	Q1	Q1	Q1	Q1							
								Q2	Q2	Q2	Q2			
Q3	Q3	Q3												
		Q4	Q4	Q4	Q4	Q4	Q4							
	Q5	Q5	Q5	Q5										

満点解法まとめ

- クエリ先読み時刻順に捌く
- 時刻0の各区画の火の強さがどこまで波及するか考える
 - 区画ごとに二分探索を行い $O(N \log N)$
- 平行四辺形のクエリを縦と斜めの線のクエリに分解する
 - 平行四辺形ごとに定数個の線に分解される
- 縦用と斜め用でデータ構造に分けてクエリを処理
 - 更新 $O(N)$ 回, 区緩和 $O(Q)$ 回
- 全体で $O((N + Q) \log N)$

得点分布



"JOI2019/2020 本選5 火事(fire)"